Route Unity WebGL players through one WSS port

A free Unity WebGL multiplayer stack: VirtualBox VM, Let's Encrypt certificates, Netcode for GameObjects, nginx and a Node.js lobby

Unity 6 · Netcode for GameObjects 2.x · Ubuntu Server · nginx · Node.js

Every code block is labelled VERIFIED, TESTED or UNTESTED. Only the Node lobby tools were actually run; the C#, nginx, systemd and VirtualBox material has not been run and must be proven on your own machines. Chapter 6 is the 10-minute spike that decides the room-routing plan.

Repository: github.com/SynokJ/multiplayer_maze

Contents

How to read this guide
Chapter 0. Corrections to earlier outputs
Chapter 1. Architecture and decisions
1.1 The shape
1.2 Decision table
1.3 What NGO's WebSocket transport can and cannot do
Chapter 2. VirtualBox VM and free HTTPS/WSS certificates
2.1 Which Ubuntu
2.2 Create the VM from the command line (UNTESTED)
2.3 NAT versus bridged
2.4 Free domain with DuckDNS, and the NAT trick
2.5 Issue the certificate: three tools
2.6 Let's Encrypt IP certificates: when they apply
2.7 Local-only options (no public name)
2.8 Base hardening and clock (UNTESTED, standard practice)
Chapter 3. Unity networking scripts: NetworkVariable, Rpc and WebSocket transport
3.1 RPC API in NGO 2.x
3.2 NetworkVariable and NetworkList
3.3 Transport settings for WebSockets and the path
3.4 Dedicated-server bootstrap with HMAC approval (UNTESTED; API names VERIFIED)
3.5 WebGL client join (UNTESTED; API names VERIFIED)
3.6 Gameplay-layer samples (UNTESTED; API names VERIFIED)
3.7 Version and prefab protection
3.8 WebGL runtime constraints
3.9 Background tabs and reconnect
Chapter 4. Building the WebGL client and the Linux dedicated server
4.1 WebGL Player Settings
4.2 Editor build script (UNTESTED; skeleton to compile and fix)
4.3 Command-line builds (flags UNTESTED; Linux server flags from Unity 6000.6 docs summary)
4.4 Test the WebGL build locally
Chapter 5. The Linux server: nginx, Node lobby, systemd and room routing
5.1 File layout
5.2 Request flow in words
5.3 nginx configuration (UNTESTED)
5.4 Caddy alternative (UNTESTED)
5.5 The Node lobby (TESTED)
5.6 Configuration file and systemd (UNTESTED)
5.7 Deploy script (UNTESTED)
5.8 Sizing
5.9 Fallback routes if the spike fails
Chapter 6. The 10-minute spike that proves path routing
6.1 Steps

Chapter 7. Testing strategy
7.1 Test ladder
7.2 Why a Node ws bot cannot play
7.3 Smoke tests (UNTESTED, standard tooling)
7.4 NGO integration tests
Chapter 8. Troubleshooting tables
8.1 Connection and TLS
8.2 Build and static files
8.3 Certificates and DNS
8.4 Process and platform
Chapter 9. Security hardening
9.1 Host baseline (UNTESTED)
9.2 Layered controls
9.3 Server-authority rules
9.4 Denial of service
Chapter 10. CI/CD
10.1 Position on Unity builds in CI
10.2 Lobby test and deploy workflow (UNTESTED)
10.3 Optional GameCI job (UNVERIFIED for Personal licenses in 2026)
Chapter 11. Pitfalls checklist
Chapter 12. Diagram index
Chapter 13. Claims ledger: what is proven, what is not
Conclusion

The free stack works on paper and its one risky assumption is now checkable: NGO 2.11.0 (19 March 2026) added `UnityTransport.ConnectionData.WebSocketPath`, so a browser client can request `wss://HOST/r/<port>` and nginx can rewrite that to `ws://127.0.0.1:<port>/` for the right room process. This overturns the earlier conclusion in this project (`reports/Free Unity WebGL multiplayer workflow.md` and `docs/WORKFLOW.md`) that path routing was "probably impossible"; that conclusion was a gap in the evidence, not a finding. The implementation is: a VirtualBox Ubuntu Server VM; a free DuckDNS name with a Let's Encrypt certificate issued by DNS-01 (so it works behind NAT or on a private LAN address with no purchased domain); nginx terminating TLS and serving the Unity WebGL build; a Node 22 / Fastify lobby that spawns one Unity Linux Dedicated Server per room on loopback ports 7000-7099; and systemd supervising the lobby (rooms are children in the same cgroup). The Node lobby was actually run (with a stand-in room server) and its create, list, join, validation, reaper and SIGTERM behaviours passed. **Everything else, including all nginx, Caddy, systemd, C# and VirtualBox material, is UNTESTED**: no Unity Editor, VirtualBox, nginx, Caddy or systemd existed in the research sandbox, and most vendor documentation hosts were unreachable, so many "docs" claims are known only through search summaries. Three claims are load-bearing and unproven until a 10-minute spike passes (Chapter 6): that the WebGL JavaScript WebSocket plugin really puts `WebSocketPath` on the wire, that a TLS-terminating proxy is transparent to Unity Transport frames (one third-party issue claims otherwise), and that the client/server encryption asymmetry (`UseEncryption = true` on the browser, `false` on the server) behaves as expected. A port-range fallback and a Node upgrade router are included, both ready to use if the spike fails.

How to read this guide

Every code block, command and config carries one of three labels. The label states how much evidence stands behind it, and it is deliberately conservative.

Label	Meaning	Examples in this guide
VERIFIED	The fact or API name was read in primary source text (GitHub raw files such as the NGO repository, the Public Suffix List, a plugin README, PyPI page). This does not mean the code compiled or ran.	NGO API names, `WebSocketPath`, `caddy-dns/duckdns` options, certbot-dns-duckdns command
TESTED	The code was actually executed in the research sandbox (Node 22.22.0).	The Node lobby, `ws-router.mjs`, `fake-room-server.mjs`
UNTESTED	Authored from standard practice or search summaries and not executed. Treat as a strong draft.	nginx, Caddy, systemd, VBoxManage, C# scripts, deploy script, CI

Two further conventions. First, "docs say" claims taken from search-result summaries rather than a fetched page are marked **(summary)**. Second, each chapter opens with a pre-flight checklist: tick every box before starting, and run the "proof" line at the end of the chapter before moving on. Placeholders to replace: `maze.example.duckdns.org` (your DuckDNS name), `maze` (VM user and service account), `you@example.com`.

The project assumptions come from the earlier workflow: Unity 6 (the notes use 6000.6.6f1 but could not verify that specific editor against NGO's declared minimum of 6000.0), Netcode for GameObjects 2.x over Unity Transport WebSockets, a headless Linux server per room, and a custom lobby instead of Unity Relay (Relay is capped at 50 average monthly CCU free, then paid) (Unity support).

Chapter 0. Corrections to earlier outputs

Earlier statement (where)	Status now	Why
"Unity's WebSocket client probably cannot send a path; path routing is expected to fail for NGO" (`docs/WORKFLOW.md` lines 15, 23, 72)	**Overturned (pending spike)**	NGO CHANGELOG 2.11.0 (2026-03-19): "Added a `WebSocketPath` field to `UnityTransport.ConnectionData` ... controls the path clients will connect to and servers/hosts will listen on when using WebSockets (#3901)" (NGO CHANGELOG). Underneath, Unity Transport has taken a path via `WithWebSocketParameters` since 2.2.0 (2023-12-01) (summary of the Transport release notes).
"Node gateway that pipes `/room/<id>` is the simpler option" (earlier report)	**Demoted to fallback**	nginx can derive the port from the URL with a regex `location` and a variable `proxy_pass`, so no per-room Node hop and no reload are needed. The Node router remains as plan C.
Earlier Unity forum claim "NGO has no path option; you need a custom `INetworkStreamDriverConstructor`" (thread, summary)	**Superseded** by NGO 2.11.0	The workaround is only needed on NGO below 2.11.
"duckdns.org public-suffix status unconfirmed; rate limits may be shared"	**Resolved**	`duckdns.org` is on the Public Suffix List (entry "DuckDNS") (PSL raw file), so each `name.duckdns.org` is its own registered domain for Let's Encrypt limits (mechanism: LE community). By contrast `sslip.io` was not found in the PSL and users hit shared-domain limits (OpenVidu thread).
"IP certificates validate only by HTTP-01 or TLS-ALPN-01" (`WORKFLOW.md`)	**Supported, secondary sources**	Caddy community, StackScale. The Let's Encrypt page itself was not read.
Old design assumed `SetConnectionData` then path	**New gotcha**	`SetConnectionData(...)` assigns a fresh `ConnectionAddressData` and does not copy `WebSocketPath`, so set the path after calling it (UnityTransport.cs). This is my reading of the source, not a compiled test.
Earlier join-token sketch passed `-token` on the command line	**Changed**	argv is visible in `ps`. The lobby passes a shared HMAC secret to the room through the `ROOM_SECRET` environment variable, and per-player tokens travel in the NGO connection-approval payload.

What did not change: a WebGL build cannot listen (UnityTransport throws "WebGL as a server is not supported by Unity Transport, outside the Editor") (UnityTransport.cs), so "client as host" must be implemented as a server-side process owning the simulation, with the first joiner holding only lobby privileges.

Chapter 1. Architecture and decisions

Figure 1. System architecture

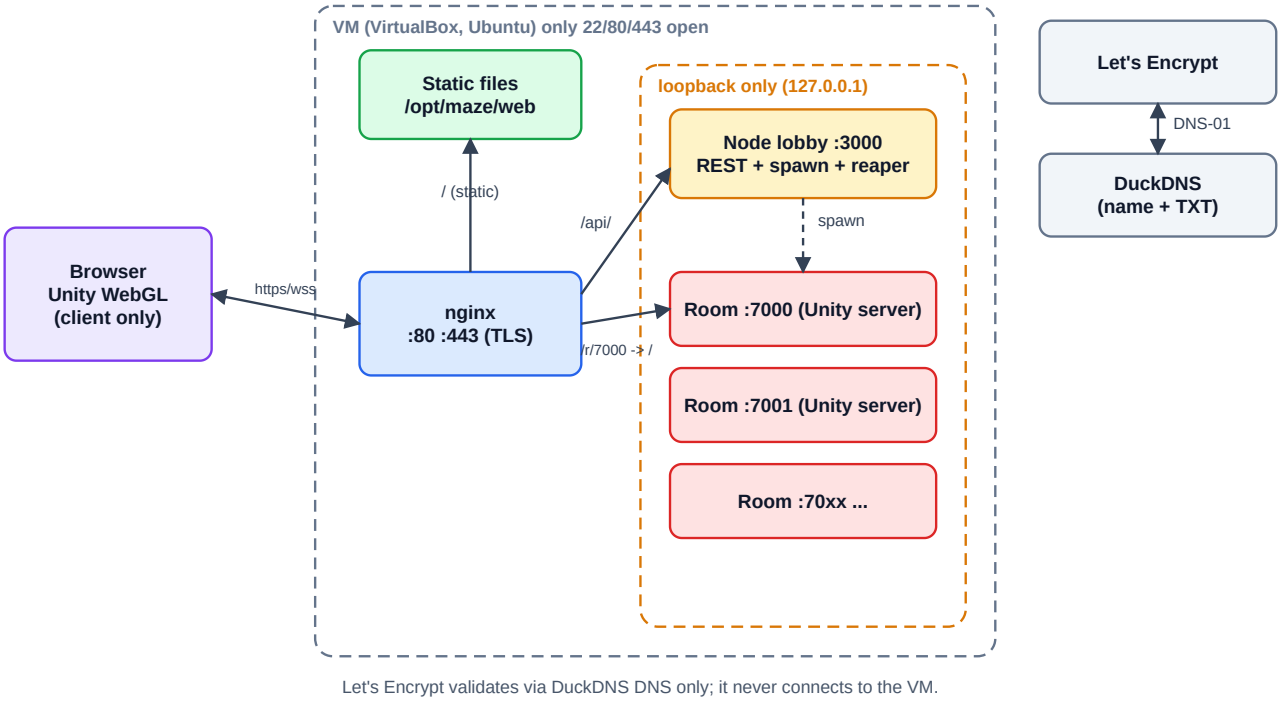
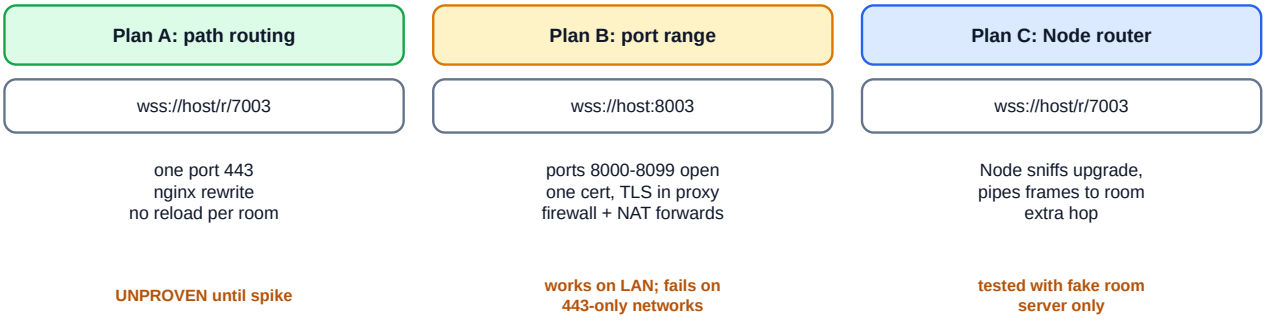


Figure 11. Room routing options compared



Pre-flight checklist

- [ ] You accept a server-authoritative design (headless Linux process per room; browsers are clients only).
- [ ] You accept Unity Personal's revenue ceiling (US$200,000 trailing 12 months) (Unity pricing updates).
- [ ] You can run an Ubuntu VM with at least 2 vCPU, 2-4 GB RAM, 20 GB disk.
- [ ] You will run the spike (Chapter 6) before investing in game logic.

1.1 The shape

The browser sees exactly one origin on port 443. Static files, the lobby REST API and every game room share it.

```
Browser (WebGL)
  |  https://HOST/                    -> nginx static   /opt/maze/web
  |  https://HOST/api/rooms           -> nginx -> Node lobby 127.0.0.1:3000
  |  wss://HOST/r/7003                -> nginx rewrite -> ws://127.0.0.1:7003/   (Unity Linux
server, room ABCDE)
VM:  nginx :80/:443   lobby :3000 (loopback)   rooms :7000-7099 (loopback)
```

Why this shape: rooms bind to loopback and never face the internet, so firewall, VirtualBox forwards and TLS collapse to ports 22/80/443. Same-origin hosting removes CORS, which is a real WebGL trap for `UnityWebRequest` (Unity docs).

1.2 Decision table

Decision	Choice	Alternative	Reason / evidence
Netcode	NGO 2.13.3 + Unity Transport 2.6.0	Mirror (SimpleWebTransport), FishNet (Bayou), Colyseus	NGO 2.13.3 requires Unity 6000.0+ and `com.unity.transport` 2.6.0 (package.json at v2.13.3). Mirror's `ClientConnect(string hostname)` has no path setting (a `ClientConnect(Uri)` overload exists), so Mirror is not better for path routing (SimpleWebTransport.cs).
TLS termination	Proxy (nginx)	Unity server terminates (`SetServerSecrets`)	Unity's manual says that without a certificate on the game server you need Relay or a TLS-terminating reverse proxy (summary of Transport 2.5 manual); a forum thread found ticking "Use Encryption" alone insufficient (Unity Discussions).
Proxy	nginx	Caddy	Port from URL regex needs no reload; fine control of `Content-Encoding`. Caddy is easier for automatic certs but DuckDNS DNS-01 needs a custom `xcaddy` build.
Certificates	certbot + `certbot-dns-duckdns` (DNS-01)	acme.sh, Caddy	Works behind NAT and on a private A record.
Room processes	One Unity process per room, child of lobby	Many rooms in one process	NGO has one `NetworkManager` per process; no source found for multi-session hosting.
Supervision	systemd, `KillMode=control-group`	Docker	Docker socket access would hand the lobby root-equivalent power.

1.3 What NGO's WebSocket transport can and cannot do

Unity states that UTP WebSocket support is "not intended as a general-purpose WebSocket library" and only connects UTP clients to UTP servers (summary of Transport 2.3 manual). Consequences: a plain Node `ws` bot cannot play the game, and a Node relay cannot speak to an NGO client. A TLS-terminating proxy that forwards frames unchanged should be transparent, but one third-party issue says a WebSocket proxy "completes the upgrade but the Unity Transport server discards the data" (PlayFab issue #9, one reporter's diagnosis, no maintainer reply). The spike settles this.

Proof for Chapter 1: you can draw the diagram in 1.1 from memory and name the three unproven assumptions.

Chapter 2. VirtualBox VM and free HTTPS/WSS certificates

Figure 2. Certificate issuance with DNS-01 (works behind NAT)

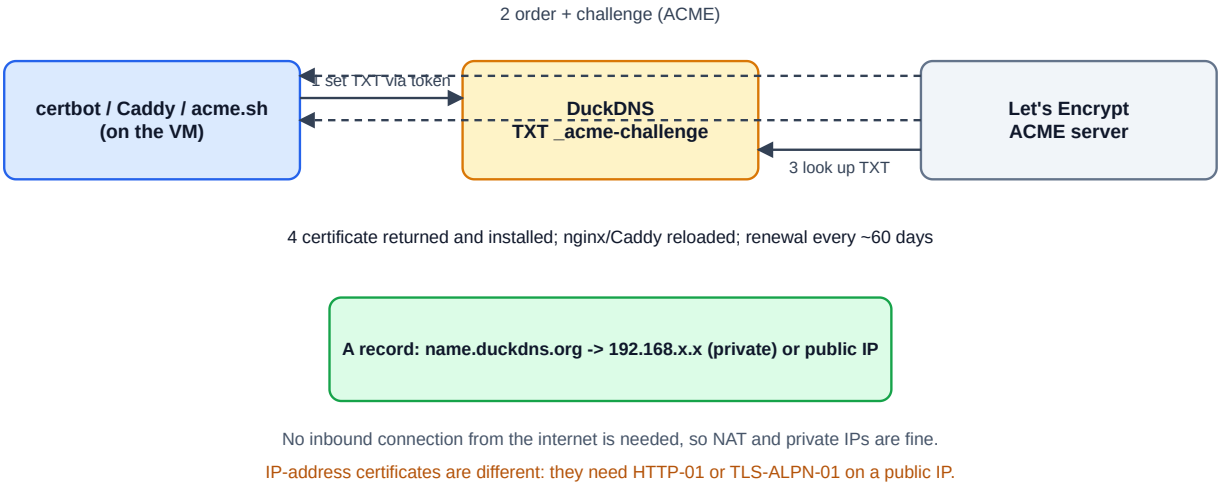
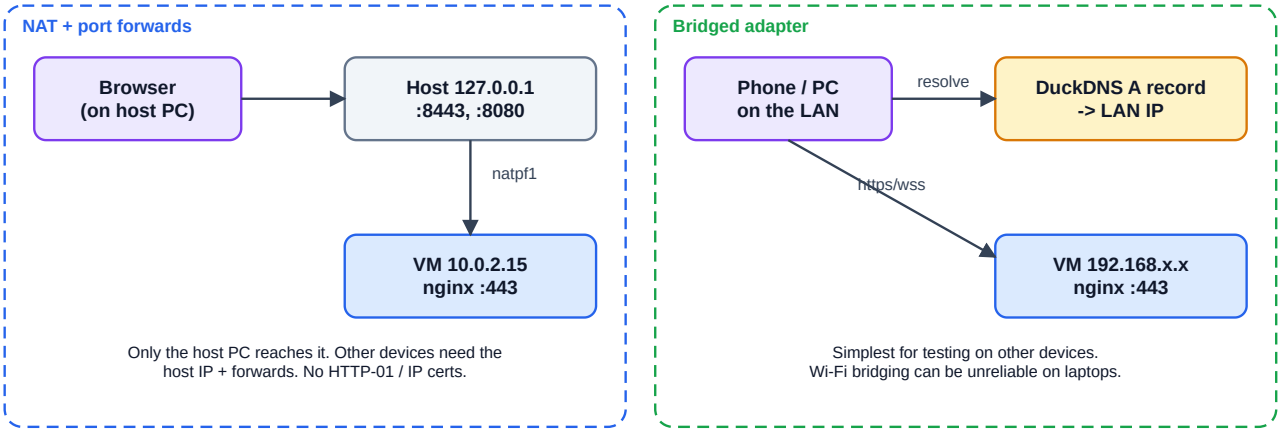


Figure 8. VirtualBox network modes



Pre-flight checklist

- [ ] VirtualBox 7.x installed on the host; hardware virtualization enabled in BIOS.
- [ ] Ubuntu Server ISO downloaded; SHA256 checked against the published `SHA256SUMS`.
- [ ] A DuckDNS account (sign in with an identity provider) and a subdomain created; token copied and stored in a password manager, never in a repo.
- [ ] You know whether the VM will be NAT or bridged (table 2.3) and whether it is reachable from the public internet (almost certainly not).
- [ ] Host firewall will allow the forwarded or bridged ports.

2.1 Which Ubuntu

Ubuntu Server 26.04 LTS ("Resolute Raccoon") was released 23 April 2026 and is supported to April 2031; server minimums are about 1.5 GB RAM and 4-5 GB disk, unchanged from 24.04 (release notes, Linuxiac, summaries). If `VBoxManage unattended detect` does not recognize the ISO, use 24.04 LTS or the GUI installer. Whether unattended install supports 26.04 was not confirmed. Guest Additions are not needed for a headless SSH-managed server (general knowledge).

2.2 Create the VM from the command line (UNTESTED)

Flag names are standard VBoxManage 7.x usage but the Oracle manual could not be fetched. `unattended install` takes `--iso` (required), `--user`, `--password`, `--hostname`, `--install-additions`, `--start-vm` (summary of Oracle docs).

```
# UNTESTED. On the host. Windows: use "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
ISO=ubuntu-26.04-live-server-amd64.iso
VM=maze-vm
VBoxManage unattended detect --iso="$ISO"          # check the guest OS is recognised first
VBoxManage createvm --name $VM --ostype Ubuntu_64 --register
VBoxManage modifyvm $VM --memory 4096 --cpus 2 --ioapic on --boot1 dvd --boot2 disk \
  --nic1 nat --natpf1 "ssh,tcp,,2222,,22" \
  --natpf1 "http,tcp,,8080,,80" --natpf1 "https,tcp,,8443,,443"
VBoxManage createmedium disk --filename "$HOME/VirtualBox VMs/$VM/$VM.vdi" --size 20480
VBoxManage storagectl $VM --name SATA --add sata --controller IntelAhci
VBoxManage storageattach $VM --storagectl SATA --port 0 --device 0 --type hdd \
  --medium "$HOME/VirtualBox VMs/$VM/$VM.vdi"
VBoxManage storagectl $VM --name IDE --add ide
VBoxManage unattended install $VM --iso="$ISO" --user=maze --password='CHANGE_ME' \
  --hostname=maze-vm.local --start-vm=headless
# If unattended install is unsupported: VBoxManage startvm $VM  and use the GUI installer
# (tick "Install OpenSSH server" in the installer).
ssh -p 2222 maze@127.0.0.1          # NAT;   bridged: ssh maze@<vm-ip>
ssh-copy-id -p 2222 maze@127.0.0.1
```

GUI route: New, name and ISO, optionally "Skip Unattended Installation", 4096 MB, 2 CPUs, 20 GB, then Settings, Network.

2.3 NAT versus bridged

Mode	VM reachable from	Needs	Good for	Caveats
NAT	Host only, unless forwards exist	Per-port forward rules; host ports under 1024 may need privileges, so use 8080/8443	Solo dev on one machine	LAN peers connect to the host IP; open the host firewall
Bridged	Whole LAN as its own host	Exact adapter name	Phones and second PCs	Wi-Fi bridging can be unreliable (general experience, unverified); exposed to the LAN
Host-only	Host and other VMs	none	Isolated tests	No phone testing

Sources: Cleverence manual summary, DevGenius.

```
# UNTESTED. NAT forwards (name,proto,hostip,hostport,guestip,guestport)
VBoxManage modifyvm maze-vm --natpf1 "https,tcp,,443,,443"     # VM powered off
VBoxManage controlvm maze-vm natpf1 "extra,tcp,,9000,,9000"    # while running
```

```
VBoxManage modifyvm maze-vm --natpf1 delete ssh
# Bridged:
VBoxManage list bridgedifs
VBoxManage modifyvm maze-vm --nic1 bridged --bridgeadapter1 "<exact adapter name>"
```

With the path-routing design only 22, 80 and 443 need forwarding. Only the port-range fallback (Chapter 5.9) needs a 100-port loop: `for p in $(seq 8000 8099); do VBoxManage modifyvm maze-vm --natpf1 "ws$p,tcp,,$p,,$p"; done` (no range syntax is known to exist; unverified).

2.4 Free domain with DuckDNS, and the NAT trick

A DuckDNS A record may point at a private address. Let's Encrypt validates DNS-01 by querying only the TXT record, so the certificate is real and browser-trusted even though the VM is not internet-reachable. This is the one sourced route to a trusted certificate for a non-public VM (Caddy community, caddy-dns/duckdns).

The DuckDNS update API takes `domains`, `token`, optional `ip`, `ipv6`, `verbose`, `clear`, and replies `OK` or `KO`; an omitted `ip` means the detected public IPv4, but any IP including a private one can be supplied; TXT uses `txt=`, and one TXT record serves the domain and all its sub-subdomains, so only one TXT exists at a time (DuckDNS spec, summary).

```
# UNTESTED, standard DuckDNS API. Point the name at the bridged VM's LAN IP.
TOKEN=...   # from the top of the duckdns.org page after login
curl -s "https://www.duckdns.org/update?domains=maze&token=$TOKEN&ip=192.168.1.50"   # expect OK
dig +short maze.duckdns.org                                                          # expect
192.168.1.50
```

Home routers with DNS-rebind protection may drop private answers; the fix is a router exception or a hosts-file entry (inference). Not confirmed by any source: the free-account subdomain limit (commonly said to be 5). No-IP free hostnames need confirmation every 30 days or so (23 active days, 7-day confirmation window, 14-day redemption), so prefer DuckDNS for long-lived development (No-IP support).

2.5 Issue the certificate: three tools

Use `--staging` until the flow works; staging certificates are untrusted by design (Let's Encrypt, certbot-dns-duckdns on PyPI). Let's Encrypt limits authorization failures to 5 per identifier per account per hour, and 5 duplicate certificates per 7 days (LE rate limits, LE community, summaries). Sources disagree on the per-registered-domain limit (20 versus 50), so check the live rate-limit page.

Option A (recommended with nginx): certbot with the DuckDNS plugin. The plugin command and credentials-file format are VERIFIED from PyPI; the venv install steps are UNTESTED.

```
# plugin usage VERIFIED; packaging steps UNTESTED
sudo apt install -y python3-venv
sudo python3 -m venv /opt/certbot
sudo /opt/certbot/bin/pip install certbot certbot-dns-duckdns          # needs certbot >= 1.18
echo 'dns_duckdns_token=TOKEN' | sudo tee /root/duckdns.ini && sudo chmod 600 /root/duckdns.ini
sudo /opt/certbot/bin/certbot certonly --non-interactive --agree-tos --email you@example.com \
  --preferred-challenges dns --authenticator dns-duckdns \
  --dns-duckdns-credentials /root/duckdns.ini --dns-duckdns-propagation-seconds 60 \
  -d maze.duckdns.org --staging          # drop --staging for the real certificate
# renewal hook and schedule
printf '#!/bin/sh\nsystemctl reload nginx\n' | sudo tee /etc/letsencrypt/renewal-
```

```
hooks/deploy/reload-nginx.sh
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
echo '0 3 * * * root /opt/certbot/bin/certbot renew -q' | sudo tee /etc/cron.d/certbot-renew
```

Plugin facts from PyPI: one DuckDNS domain per call; default propagation 30 s (examples use 60); wildcard `-d "*.maze.duckdns.org"` is supported; install as root if certbot runs as root; the snap certbot needs the snap plugin. Wildcard and apex cannot be validated in one run because DuckDNS holds a single TXT; request them separately (LE community).

Option B: acme.sh. The hook reads the environment variable `DuckDNS_Token` and `--install-cert` with `--reloadcmd` is the documented way to place files (dns_duckdns.sh, README). UNTESTED assembly:

```
curl https://get.acme.sh | sh -s email=you@example.com; source ~/.bashrc
acme.sh --set-default-ca --server letsencrypt        # some versions default to ZeroSSL
(unverified)
export DuckDNS_Token="TOKEN"
acme.sh --issue --dns dns_duckdns -d maze.duckdns.org --staging     # remove --staging when it
works
sudo mkdir -p /etc/ssl/maze
sudo acme.sh --install-cert -d maze.duckdns.org \
  --fullchain-file /etc/ssl/maze/fullchain.pem --key-file /etc/ssl/maze/privkey.pem \
  --reloadcmd "systemctl reload nginx"
```

Option C: Caddy with the DuckDNS module. Caddy issues and renews automatically, but needs a custom build: `xcaddy build --with github.com/caddy-dns/duckdns`; without it you get "module not registered: dns.providers.duckdns"; check with `caddy list-modules`. The module name is `dns.providers.duckdns`, Caddyfile `tls { dns duckdns {env.DUCKDNS_API_TOKEN} }`, options `override_domain` and `resolver` (default 8.8.8.8:53) (caddy-dns/duckdns README, VERIFIED; build advice from Caddy community, summary). A 2025 log showed module v0.4.0, and bug reports exist of the plugin using the wrong zone (Caddy community, summary).

```
# UNTESTED
sudo apt install -y golang-go
go install github.com/caddyserver/xcaddy/cmd/xcaddy@latest
~/go/bin/xcaddy build --with github.com/caddy-dns/duckdns
sudo apt install -y caddy && sudo systemctl stop caddy
sudo cp caddy /usr/bin/caddy && caddy list-modules | grep duckdns
sudo setcap cap_net_bind_service=+ep /usr/bin/caddy       # if the copied binary cannot bind 80/443
echo 'DUCKDNS_API_TOKEN=xxxx' | sudo tee /etc/caddy/duckdns.env && sudo chmod 600
/etc/caddy/duckdns.env
sudo systemctl edit caddy      # add:  [Service]  EnvironmentFile=/etc/caddy/duckdns.env
```

2.6 Let's Encrypt IP certificates: when they apply

IP-address and short-lived certificates went generally available on 15 January 2026; IP certificates are always the 160-hour "shortlived" profile, for IPv4 or IPv6, and validate by HTTP-01 or TLS-ALPN-01 only (Let's Encrypt GA post (summary), Help Net Security, StackScale). They need ports 80 or 443 reachable from the public internet, so they do **not** work on a NAT or LAN VM unless you forward the ports at the router and the ISP gives a reachable public IPv4 (no CGNAT); they do work on a public free VM such as Oracle Cloud. Certbot supports them from 5.3 (`--ip-address`) and webroot IPs from 5.4; the official command is below (Let's Encrypt, EFF, summaries). The acme.sh README has no mention of IP certificates (VERIFIED absence).

```
# UNTESTED here; command text from the Let's Encrypt/EFF announcement (summary). Public IP only.
sudo certbot certonly --staging --preferred-profile shortlived --webroot --webroot-path
```

```
/var/www/acme --ip-address 203.0.113.10
```

```
# UNTESTED. Caddy profile syntax from Caddy community; the Dec 2025 caddy#7399 issue predates GA
and is probably obsolete
https://203.0.113.10 {
    tls { issuer acme { profile shortlived } }
    reverse_proxy 127.0.0.1:7777
}
```

Renewal must run at least daily because of the 160-hour life; Caddy reportedly renews about every 2 days.

2.7 Local-only options (no public name)

Caddy `tls internal` or mkcert give certificates from a local CA, which every client device must trust. Self-signed certificates do not work in browsers by default (Transport manual, summary) and WebSocket failures on untrusted certificates are silent, so open `https://HOST` in a tab and accept it first, or install the CA. All trust procedures below are UNTESTED and unsourced.

```
mkcert -install && mkcert 192.168.1.50 mazevm.local            # mkcert (sudo apt install mkcert
libnss3-tools)
# Caddy root CA: /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt
certutil -addstore -f ROOT root.crt                            # Windows (admin)
sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain root.crt   #
macOS
```

Android: Settings, Security, Install certificate, CA certificate (Chrome accepts user CAs on Android 7+, apps may not). iOS: install the profile, then Settings, General, About, Certificate Trust Settings. Firefox has its own store (`security.enterprise_roots.enabled`).

2.8 Base hardening and clock (UNTESTED, standard practice)

```
sudo apt update && sudo apt -y upgrade
sudo ufw default deny incoming; sudo ufw default allow outgoing
sudo ufw allow OpenSSH; sudo ufw allow 80/tcp; sudo ufw allow 443/tcp
sudo ufw enable
sudo apt install -y unattended-upgrades fail2ban && sudo dpkg-reconfigure -plow unattended-upgrades
timedatectl status          # want: System clock synchronized: yes (suspended VMs drift; TLS and
ACME need correct time)
sudo useradd --system --home /opt/maze --shell /usr/sbin/nologin maze
```

Proof for Chapter 2: `dig +short maze.duckdns.org` returns the expected IP; `sudo openssl x509 -in /etc/letsencrypt/live/maze.duckdns.org/fullchain.pem -noout -issuer -dates` shows the (staging, then production) issuer and dates; `ssh` key login works.

Chapter 3. Unity networking scripts: NetworkVariable, Rpc and WebSocket transport

Pre-flight checklist

- [ ] Unity 6 project (declared NGO minimum 6000.0; your 6000.6.6f1 is untested against it).
- [ ] `Packages/manifest.json` pins `"com.unity.netcode.gameobjects": "2.13.3"` (and optionally `"com.unity.transport": "2.6.0"`). Version 2.13.3 is the latest tag seen, dated 2026-09-14 in the CHANGELOG; the Package Manager may offer newer (package.json, CHANGELOG). Require at least 2.11.0 for `WebSocketPath`.
- [ ] After install, `Packages/packages-lock.json` shows `com.unity.transport` 2.6.x.
- [ ] Scene has a `NetworkManager` with `UnityTransport`, a player prefab with `NetworkObject`, and a Network Prefabs List (Create, Netcode, Network Prefabs List) assigned to the manager.
- [ ] "Use Web Sockets" ticked on the transport (the inspector then also shows the WebSocket Path).

Honesty note. All API names below were read from the NGO source (develop-2.0.0 branch plus tag v2.13.3); **no sample was compiled** because no Unity Editor existed in the research environment. Treat each C# block as VERIFIED for API names and UNTESTED for compile and behaviour. Transport-internal behaviour (server path strictness, wss scheme mapping) is known only from documentation summaries because `com.unity.transport` source is not on GitHub.

3.1 RPC API in NGO 2.x

Use the unified `[Rpc(SendTo.X)]`. `[ServerRpc]` and `[ClientRpc]` still exist as legacy thin subclasses (ServerRpc is `SendTo.Server`, ClientRpc is `SendTo.NotServer`) and `RequireOwnership` is obsolete in favour of `InvokePermission` (RpcAttributes.cs). Method names must end in `Rpc`, or the IL post-processor errors with "Rpc method must end with 'Rpc' suffix!" (NetworkBehaviourILPP.cs).

`SendTo`	Executes on	Notes
`Server`	the server (runs locally if called on server)	any client may call it unless `InvokePermission` restricts
`NotServer`	all non-server peers, observer-filtered	not sent to a host-mode server
`ClientsAndHost`	same as `Everyone` in host mode; same as `NotServer` on a dedicated server	the right broadcast on a dedicated server
`Everyone`	all peers including the server, observer-filtered	runs locally for the caller
`Owner`, `NotOwner`, `Me`, `NotMe`, `Authority`, `NotAuthority`	by ownership/authority	
`SpecifiedInParams`	targets from `RpcSendParams`	needs a runtime target

Source: RpcTarget.cs. Attribute fields: `Delivery` (Reliable default or Unreliable), `InvokePermission` (`Everyone`=0, `Server`, `Owner`), `DeferLocal`, `AllowTargetOverride`. The default being `Everyone` is inferred from enum ordering and must be confirmed in the Editor; **so set `InvokePermission = RpcInvokePermission.Owner` explicitly on anything that should only be called by the owner**. Receiver

identity comes from a trailing `RpcParams rpcParams = default` parameter: `rpcParams.Receive.SenderClientId`. Never trust a client-supplied id (RpcParams.cs).

3.2 NetworkVariable and NetworkList

`NetworkVariable<T>(T value = default, NetworkVariableReadPermission readPerm = Everyone, NetworkVariableWritePermission writePerm = Server)`, with `OnValueChanged(previous, next)` (NetworkVariable.cs). Read permission is `Everyone` or `Owner`; write is `Server` or `Owner`. `NetworkList<T>` requires `unmanaged, IEquatable<T>` and raises `OnListChanged` (NetworkList.cs). Use `FixedString32Bytes` rather than `string`. Lifecycle hooks: `OnNetworkSpawn`, `OnNetworkPostSpawn`, `OnNetworkDespawn`. Spawning: `NetworkObject.Spawn`, `SpawnWithOwnership`, `SpawnAsPlayerObject`, `Despawn`, `ChangeOwnership`. `NetworkConfig.TickRate` defaults to 30. Practical rules (inference): create `NetworkList` in `Awake` or a field initializer, not in `OnNetworkSpawn`; subscribe in `OnNetworkSpawn` and unsubscribe in `OnNetworkDespawn`; write server-owned variables only under `IsServer`.

3.3 Transport settings for WebSockets and the path

Facts from `UnityTransport.cs` (VERIFIED):

Setting	Fact
`ConnectionData.Address`	Accepts an IP or a domain name. No `wss://` URL is accepted.
`ConnectionData.WebSocketPath`	Default `"/"`. Tooltip: "Path to connect to or listen on when using WebSockets." Applied only when `UseWebSockets` is true, not Relay, and the string is non-blank, via `settings.WithWebSocketParameters(path: ...)`.
`SetConnectionData(...)`	Reassigns `ConnectionData = new ConnectionAddressData{...}`; does not copy `WebSocketPath`. Set the path afterwards. Also applies `-port` and `-ip` from the command line unless `forceOverrideCommandLineArgs` is true.
`UseEncryption`, `SetServerSecrets(cert, key)`, `SetClientSecrets(serverCommonName, caCertificate = null)`	TLS for WebSockets. A null CA means the OS or browser store is used. Throws if secrets are missing at start.
WebGL hosting	`StartServer` throws on WebGL outside the Editor. On WebGL the WebSocket interface is always used, with a warning if `UseWebSockets` is false.
Linux server build	WebSockets allowed (the `UNITY_SERVER` warning is only about Relay).

Server strictness: the Transport manual (summary) says clients send all requests to the configured path and that "on the server side, it will only allow connections made to the given path" (Transport 2.5 manual). That is why the proxy rewrites `/r/70xx` to `/`, the server's default path. Query strings in the path have no supported documentation; do not use them. Carry the room token in the connection-approval payload instead.

The encryption asymmetry (UNVERIFIED). With the proxy terminating TLS, the browser must use `wss` while the server must listen on plain `ws`. The plan is `UseEncryption = true` plus `SetClientSecrets(host)` on the client and `UseEncryption = false` on the server. Whether the client's `UseEncryption = true` yields a `wss://` URL on WebGL was not verified from source; the spike

confirms it. Unity's manual says the client needs only the server name (`WithSecureClientParameters(serverName)`) and trust comes from the browser store.

3.4 Dedicated-server bootstrap with HMAC approval (UNTESTED; API names VERIFIED)

This merges the research's two samples. It matches the lobby in Chapter 5: the lobby passes `-port`, `-roomId`, `-maxPlayers` on the command line and `ROOM_SECRET`, `LOBBY_URL`, `LOBBY_INTERNAL_KEY` through the environment; the player token is `base64url(JSON{r,n,j,exp}).base64url(HMAC-SHA256(secret, payloadB64))`.

```
// Assets/Scripts/Net/ServerBootstrap.cs   (UNTESTED; compile in Unity)
#if UNITY_SERVER
using System;
using System.Collections;
using System.Collections.Generic;
using System.Security.Cryptography;
using System.Text;
using Unity.Netcode;
using Unity.Netcode.Transports.UTP;
using UnityEngine;
using UnityEngine.Networking;

[Serializable] public class JoinClaims { public string r; public string n; public string j; public
long exp; }

public class ServerBootstrap : MonoBehaviour
{
    string roomId; byte[] secret; int maxPlayers = 8;
    readonly HashSet<string> usedJti = new HashSet<string>();
    string lobbyUrl, internalKey;

    static string Arg(string name, string fallback)
    {
        var a = Environment.GetCommandLineArgs();
        for (int i = 0; i < a.Length - 1; i++) if (a[i] == name) return a[i + 1];
        return fallback;
    }
    static byte[] B64UrlDecode(string s)
    {
        s = s.Replace('-', '+').Replace('_', '/');
        switch (s.Length % 4) { case 2: s += "=="; break; case 3: s += "="; break; }
        return Convert.FromBase64String(s);
    }

    void Start()
    {
        Application.targetFrameRate = 30;              // headless loops otherwise spin a core
        ushort port = ushort.Parse(Arg("-port", "7777"));
        roomId     = Arg("-roomId", "DEV00");
        maxPlayers = int.Parse(Arg("-maxPlayers", "8"));
        secret     = Encoding.UTF8.GetBytes(Environment.GetEnvironmentVariable("ROOM_SECRET") ??
"dev-secret-dev-secret-dev-secret!");
        lobbyUrl   = Environment.GetEnvironmentVariable("LOBBY_URL");
        internalKey= Environment.GetEnvironmentVariable("LOBBY_INTERNAL_KEY");

        var nm  = NetworkManager.Singleton;
        var utp = (UnityTransport)nm.NetworkConfig.NetworkTransport;
        utp.UseWebSockets = true;
        utp.UseEncryption = false;                     // TLS is terminated by nginx
        // true = do not let UTP re-read -port/-ip itself; 127.0.0.1 keeps the room off the network
        utp.SetConnectionData(true, "127.0.0.1", port, "127.0.0.1");
        var cd = utp.ConnectionData; cd.WebSocketPath = "/"; utp.ConnectionData = cd;   // AFTER
SetConnectionData

        nm.NetworkConfig.ConnectionApproval = true;
        nm.ConnectionApprovalCallback = Approve;
```

```
        nm.OnServerStarted += () => { Debug.Log("ROOM_READY " + roomId + " " + port);
StartCoroutine(Heartbeat()); };
        nm.OnClientDisconnectCallback += id => Debug.Log("left " + id);
        if (!nm.StartServer()) { Debug.LogError("StartServer failed"); Application.Quit(1); }
    }

    void Approve(NetworkManager.ConnectionApprovalRequest req,
NetworkManager.ConnectionApprovalResponse res)
    {
        res.Approved = false; res.CreatePlayerObject = true; res.Pending = false;
        try
        {
            var tok = Encoding.UTF8.GetString(req.Payload).Split('.');
            if (tok.Length != 2) { res.Reason = "bad token"; return; }
            using var h = new HMACSHA256(secret);
            var expect = h.ComputeHash(Encoding.UTF8.GetBytes(tok[0]));
            if (!CryptographicOperations.FixedTimeEquals(expect, B64UrlDecode(tok[1]))) {
res.Reason = "bad token"; return; }
            var c = JsonUtility.FromJson<JoinClaims>
(Encoding.UTF8.GetString(B64UrlDecode(tok[0])));
            if (c.r != roomId)                                  { res.Reason = "wrong room";
return; }
            if (c.exp < DateTimeOffset.UtcNow.ToUnixTimeSeconds()) { res.Reason = "token expired";
return; }
            if (!usedJti.Add(c.j))                              { res.Reason = "token reused";
return; }
            if (NetworkManager.Singleton.ConnectedClientsIds.Count >= maxPlayers) { res.Reason =
"room full"; return; }
            res.Approved = true;
        }
        catch (Exception e) { Debug.LogWarning("approval error: " + e.Message); res.Reason = "bad
token"; }
    }

    IEnumerator Heartbeat()                                 // lobby reaps rooms that stop
reporting
    {
        var wait = new WaitForSeconds(5f);
        while (NetworkManager.Singleton.IsServer)
        {
            if (!string.IsNullOrEmpty(lobbyUrl))
            {
                var body = "{\"players\":" + NetworkManager.Singleton.ConnectedClientsIds.Count +
"}";
                using var www = new UnityWebRequest(lobbyUrl + "/internal/rooms/" + roomId +
"/heartbeat", "POST")
                {
                    uploadHandler = new UploadHandlerRaw(Encoding.UTF8.GetBytes(body)),
                    downloadHandler = new DownloadHandlerBuffer()
                };
                www.SetRequestHeader("Content-Type", "application/json");
                www.SetRequestHeader("x-internal-key", internalKey);
                yield return www.SendWebRequest();
            }
            yield return wait;
        }
    }

    void OnApplicationQuit() { if (NetworkManager.Singleton) NetworkManager.Singleton.Shutdown(); }
}
#endif
```

Notes. The lobby sends SIGTERM to stop rooms; whether Unity routes SIGTERM into a clean `OnApplicationQuit` was not tested, and the lobby falls back to SIGKILL after 5 s. Call `Shutdown()` then wait for `OnServerStopped` before `Application.Quit()` if you add an in-game stop command. The server should use the HMAC claims `n` (player name) as the display name.

3.5 WebGL client join (UNTESTED; API names VERIFIED)

```
// Assets/Scripts/Net/ClientConnect.cs   (UNTESTED)
using System;
using System.Text;
using Unity.Netcode;
using Unity.Netcode.Transports.UTP;
using UnityEngine;

public class ClientConnect : MonoBehaviour
{
    // Values come from the lobby JSON: host, port (443), path ("/r/7003"), secure (true), token
    public void Join(string host, ushort port, string path, bool secure, string token)
    {
        Application.runInBackground = true;
        var nm  = NetworkManager.Singleton;
        var utp = (UnityTransport)nm.NetworkConfig.NetworkTransport;
        utp.UseWebSockets = true;
        utp.UseEncryption = secure;                       // wss (mapping to scheme UNVERIFIED: the
spike proves it)
        if (secure) utp.SetClientSecrets(host);           // common name = host name; null CA =
browser trust store
        utp.SetConnectionData(host, port);
        var cd = utp.ConnectionData; cd.WebSocketPath = path; utp.ConnectionData = cd;   // AFTER
SetConnectionData

        nm.NetworkConfig.ConnectionApproval = true;
        nm.NetworkConfig.ConnectionData = Encoding.UTF8.GetBytes(token + "");   // token is the
approval payload
        nm.OnClientDisconnectCallback += id =>
        {
            if (id == nm.LocalClientId) Debug.Log("disconnected: " + nm.DisconnectReason);
        };
        if (!nm.StartClient()) Debug.LogError("StartClient failed");
    }
}
```

If the compiler lets you assign `utp.ConnectionData.WebSocketPath = path` directly (the earlier notes saw it as a public field and, elsewhere, as a struct property), use whichever compiles; the copy-modify-assign form always works. Fetching the lobby JSON from WebGL uses `UnityWebRequest.Post("/api/rooms", ...)` against the same origin (relative URLs resolve to the page origin).

3.6 Gameplay-layer samples (UNTESTED; API names VERIFIED)

These cover synced variables and remote calls without prescribing the game. Player prefab state:

```
using Unity.Collections;
using Unity.Netcode;
using UnityEngine;

public class PlayerState : NetworkBehaviour
{
    public NetworkVariable<int> Score = new NetworkVariable<int>(0);
    public NetworkVariable<Color> Tint = new NetworkVariable<Color>(Color.white);
    public NetworkVariable<FixedString32Bytes> PlayerName = new NetworkVariable<FixedString32Bytes>
("Player");

    public override void OnNetworkSpawn()
    {
        Score.OnValueChanged += OnScore;
        Tint.OnValueChanged  += (_, c) => GetComponent<Renderer>().material.color = c;
        GetComponent<Renderer>().material.color = Tint.Value;
        if (IsServer)
        {
```

```
            Tint.Value = Color.HSVToRGB(Random.value, 0.7f, 1f);
            PlayerName.Value = $"P{OwnerClientId}";
        }
    }
    public override void OnNetworkDespawn() { Score.OnValueChanged -= OnScore; }
    void OnScore(int prev, int next) => Debug.Log($"{PlayerName.Value} score {prev}->{next}");
    public void AddScore(int n) { if (IsServer) Score.Value += n; }

    [Rpc(SendTo.Server, InvokePermission = RpcInvokePermission.Owner)]
    public void SetNameRpc(FixedString32Bytes n, RpcParams p = default)
    {
        if (p.Receive.SenderClientId == OwnerClientId) PlayerName.Value = n;
    }
}
```

Chat via server relay (`SendTo.ClientsAndHost` behaves like `NotServer` on a dedicated server):

```
using Unity.Collections;
using Unity.Netcode;
using UnityEngine;

public class Chat : NetworkBehaviour
{
    [Rpc(SendTo.Server)]
    public void SendChatRpc(FixedString128Bytes msg, RpcParams p = default)
    {
        if (msg.Length == 0) return;                        // validate: length, rate, content
        BroadcastChatRpc(p.Receive.SenderClientId, msg);
    }
    [Rpc(SendTo.ClientsAndHost)]
    void BroadcastChatRpc(ulong from, FixedString128Bytes msg) => Debug.Log($"[{from}] {msg}");
}
```

Server-authoritative input (client sends intent, server moves; add a server-authoritative `NetworkTransform` to replicate position):

```
using Unity.Netcode;
using UnityEngine;

public class MoverNet : NetworkBehaviour
{
    Vector2 input; public float speed = 4f;
    void Update()
    {
        if (IsOwner)
        {
            var v = new Vector2(Input.GetAxisRaw("Horizontal"), Input.GetAxisRaw("Vertical"));
            if (v != input) { input = v; SendInputRpc(v); }
        }
        if (IsServer)
            transform.position += new Vector3(input.x, 0, input.y).normalized * speed *
Time.deltaTime;
    }
    [Rpc(SendTo.Server, InvokePermission = RpcInvokePermission.Owner, Delivery =
RpcDelivery.Reliable)]
    void SendInputRpc(Vector2 v) => input = Vector2.ClampMagnitude(v, 1f);
}
```

Ready-up and phase state with a `NetworkList`, and a custom serializable struct:

```
using Unity.Netcode;
using UnityEngine;

public enum GamePhase : byte { Lobby, Countdown, Playing, Ended }

public class GameFlow : NetworkBehaviour      // place on a scene NetworkObject (in-scene objects
spawn with the server)
```

```
{
    public NetworkVariable<GamePhase> Phase = new NetworkVariable<GamePhase>(GamePhase.Lobby);
    public NetworkList<ulong> ReadyIds;
    void Awake() { ReadyIds = new NetworkList<ulong>(); }       // create in Awake, never in
OnNetworkSpawn

    public override void OnNetworkSpawn()
    {
        Phase.OnValueChanged += (a, b) => Debug.Log($"phase {a}->{b}");
        if (IsServer) NetworkManager.OnClientDisconnectCallback += id => ReadyIds.Remove(id);
    }
    [Rpc(SendTo.Server)]
    public void SetReadyRpc(bool ready, RpcParams p = default)
    {
        var id = p.Receive.SenderClientId;
        if (Phase.Value != GamePhase.Lobby) return;
        if (ready && !ReadyIds.Contains(id)) ReadyIds.Add(id);
        else if (!ready) ReadyIds.Remove(id);
        if (ReadyIds.Count >= 1 && ReadyIds.Count == NetworkManager.ConnectedClientsIds.Count)
            Phase.Value = GamePhase.Playing;
    }
}

public struct ScoreEntry : INetworkSerializable, System.IEquatable<ScoreEntry>   // shape from NGO
docs knowledge, not re-read in source
{
    public ulong ClientId; public int Score;
    public void NetworkSerialize<T>(BufferSerializer<T> s) where T : IReaderWriter
    { s.SerializeValue(ref ClientId); s.SerializeValue(ref Score); }
    public bool Equals(ScoreEntry o) => ClientId == o.ClientId && Score == o.Score;
}
```

Supported RPC parameter types are unmanaged primitives, enums, vectors, `FixedString*`, arrays of those, `INetworkSerializable` types, `NetworkObjectReference` and strings; this list comes from general NGO knowledge and was not verified (docs unreachable).

3.7 Version and prefab protection

Add a build version to the approval payload or reject on mismatch; stale browser caches serving an old client against a new server are the commonest cause of mysterious breakage (general knowledge). NGO's `NetworkConfig.ForceSamePrefabs` (default true, from memory) rejects mismatched prefab lists. Stamp the git SHA into `PlayerSettings.bundleVersion` during the build (Chapter 4) and compare `Application.version` in the approval callback (UNTESTED).

3.8 WebGL runtime constraints

Constraint	Consequence
No listening sockets, no host mode	Dedicated server or editor host only (NGO #4111: WebGL host fails for Editor clients)
No raw TCP/UDP, no threads	WebSocket transport only; UTP uses a JavaScript bridge
https page cannot open `ws://`	Mixed content block; must use `wss://` (Unity forum)
Hidden tabs throttled to about 1 Hz	See 3.9
`Application.runInBackground`	Set true (also `PlayerSettings.runInBackground`), but browsers still throttle

3.9 Background tabs and reconnect

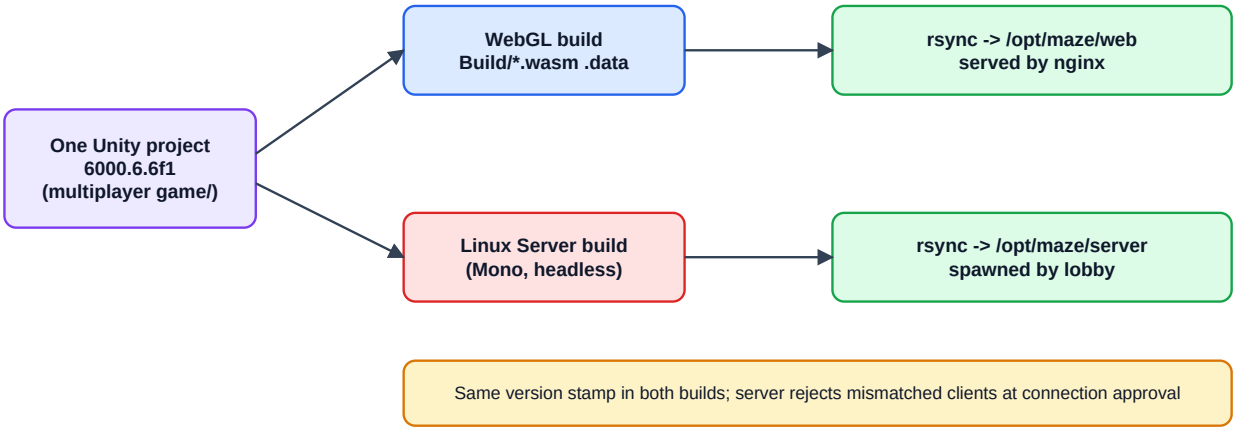
Hidden tabs update about once per second and `Time.time` runs slow even with Run In Background on (Unity WebGL performance, summary). Chrome 88 "intensive throttling" can reduce timers to once per minute after 5+ minutes hidden with a chain count of 5 or more and 30 s of silence (Chrome blog, summary). Field reports show WebSocket clients dropping after roughly 5-6 minutes in a hidden Chrome tab (Solace) and a Unity Relay session failing after about 2 minutes (Unity issue tracker, marked Won't Fix). Whether Chrome still exempts tabs with active WebSockets is unconfirmed.

Design rules (inference): set the Unity Transport heartbeat and disconnect timeouts generously (90-120 s; exact property names such as `HeartbeatTimeoutMS` and `DisconnectTimeoutMS` exist in `UnityTransport` but were not listed or verified); never kick on a missed client ping alone; hold a slot for 30-60 s with a rejoin token; on `visibilitychange` back to visible, check state and reconnect plus full resync; reconnect means `Shutdown()`, wait for `ShutdownInProgress == false`, then `StartClient()` with a fresh token. Prefer turn-tolerant gameplay. Test by hiding a tab for 10 minutes in each browser.

Proof for Chapter 3: the project compiles for both the WebGL and Linux Server targets; in the Editor, one server and one client connect with the HMAC token generated by the lobby test helper; a wrong token shows `DisconnectReason` "bad token".

Chapter 4. Building the WebGL client and the Linux dedicated server

Figure 7. Build and deploy pipeline



Pre-flight checklist

- [ ] Unity Hub modules installed: WebGL Build Support; Linux Dedicated Server Build Support; Linux Build Support (Mono). Add Linux Build Support (IL2CPP) only if you want IL2CPP (sysroot toolchain; manual, summary).
- [ ] Decision: Mono for the server (avoids the open IL2CPP link bug UUM-121029 reproduced on 2022.3.67f1 and 6000.x betas; fixed status in 6000.6 unknown (Unity issue tracker, summary)). WebGL is always IL2CPP.
- [ ] Scene list shared by both builds; `link.xml` present if stripping removes RPC-reached code.
- [ ] No second Editor instance open on the project (project lock).
- [ ] The two builds come from the same commit.

4.1 WebGL Player Settings

`PlayerSettings.WebGL` exposes `compressionFormat`, `decompressionFallback`, `initialMemorySize`, `maximumMemorySize`, `memoryGrowthMode`, `dataCaching`, `nameFilesAsHashes` (scripting reference, summary). Gzip is the default (bigger than Brotli, faster to build, natively supported over HTTP and HTTPS); the server must send a matching `Content-Encoding`; Decompression Fallback embeds a JavaScript decompressor for servers that cannot (Deploy a web application, summary). Recommended values below are my suggestions, not documented defaults.

Setting	Recommended	Why
Compression Format	**Gzip** to start; Brotli later (HTTPS only, from memory)	Fewest surprises; Brotli breaks on itch.io
Decompression Fallback	On while diagnosing, off once headers are verified	Bigger loader
Name Files As Hashes	On	Long immutable caching of `Build/`

Setting	Recommended	Why
Data Caching	On (off while iterating on builds)	IndexedDB cache can mask a new build
Run In Background	On	Keeps transport ticking in unfocused tabs
Managed Stripping Level	Low or Medium	NGO uses code generation and reflection; add `link.xml` if High breaks it
Exception Support	Explicitly thrown only	Smaller, faster
Memory	initial 64 MB, max 512 MB, geometric growth	Guess for a small game; tune with the profiler
WebAssembly 2023	Off	Older Safari compatibility (unverified)

Unity 6 emits WebAssembly only. Custom templates live under `Assets/WebGLTemplates/<Name>/index.html` (from memory).

Output layout (Gzip build): `index.html`, `Build/<name>.loader.js` (uncompressed), `<name>.framework.js.gz`, `<name>.data.gz`, `<name>.wasm.gz`, `TemplateData/`. Brotli uses `.br`. Whether Unity 6 ever uses a `.unityweb` suffix was not confirmed.

4.2 Editor build script (UNTESTED; skeleton to compile and fix)

```
// Assets/Editor/BuildScript.cs
using System;
using System.IO;
using UnityEditor;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEngine;

public static class BuildScript
{
    static string Version => Environment.GetEnvironmentVariable("BUILD_VERSION") ??
PlayerSettings.bundleVersion;
    static string[] Scenes()
    {
        var l = new System.Collections.Generic.List<string>();
        foreach (var s in EditorBuildSettings.scenes) if (s.enabled) l.Add(s.path);
        return l.ToArray();
    }

    [MenuItem("Build/WebGL")]
    public static void BuildWebGL() => Run(BuildTarget.WebGL, false, Path.Combine("Builds",
"WebGL", Version));
    [MenuItem("Build/Linux Server")]
    public static void BuildLinuxServer() => Run(BuildTarget.StandaloneLinux64, true,
        Path.Combine("Builds", "LinuxServer", Version, "MazeServer.x86_64"));

    // Run ONE target per Unity invocation: exiting inside Run would otherwise skip the second
build.
    static void Run(BuildTarget target, bool server, string output)
    {
        Directory.CreateDirectory(server ? Path.GetDirectoryName(output) : output);
        if (target == BuildTarget.WebGL)
        {
            PlayerSettings.WebGL.compressionFormat = WebGLCompressionFormat.Gzip;
            PlayerSettings.WebGL.decompressionFallback = false;
            PlayerSettings.WebGL.nameFilesAsHashes = true;
            PlayerSettings.WebGL.dataCaching = true;
            PlayerSettings.runInBackground = true;
        }
        var opts = new BuildPlayerOptions
        {
            scenes = Scenes(), locationPathName = output, target = target,
```

```
            targetGroup = server ? BuildTargetGroup.Standalone : BuildTargetGroup.WebGL,
            options = BuildOptions.None
        };
        if (server)
        {
            EditorUserBuildSettings.standaloneBuildSubtarget = StandaloneBuildSubtarget.Server;
            opts.subtarget = (int)StandaloneBuildSubtarget.Server;
        }
        var report = BuildPipeline.BuildPlayer(opts);
        bool ok = report.summary.result == BuildResult.Succeeded;
        Debug.Log(ok ? $"Build OK {target} -> {output} ({report.summary.totalSize} bytes)" : "Build
failed: " + report.summary.result);
        if (Application.isBatchMode) EditorApplication.Exit(ok ? 0 : 1);
        else if (!ok) throw new Exception("Build failed");
    }
}
```

`NamedBuildTarget.Server` with `PlayerSettings.SetScriptingBackend(NamedBuildTarget.Server, ScriptingImplementation.Mono2x)` is the newer API for backend selection (from memory). `BuildPlayerOptions.subtarget` is an int in 2021.2+ (from memory).

4.3 Command-line builds (flags UNTESTED; Linux server flags from Unity 6000.6 docs summary)

`-buildTarget Linux64 -standaloneBuildSubtarget Server` builds the Dedicated Server, and Unity defines `UNITY_SERVER` for it (Dedicated Server build, 6000.6, summary).

```
# Linux server (UNTESTED)
/opt/unity/6000.6.6f1/Editor/Unity -batchmode -nographics -quit \
  -projectPath "/path/to/multiplayer game" \
  -buildTarget Linux64 -standaloneBuildSubtarget Server \
  -executeMethod BuildScript.BuildLinuxServer -logFile build-server.log

# WebGL (UNTESTED; do not add -nographics, it can fail for WebGL, unverified)
/opt/unity/6000.6.6f1/Editor/Unity -batchmode -quit \
  -projectPath "/path/to/multiplayer game" -buildTarget WebGL \
  -executeMethod BuildScript.BuildWebGL -logFile build-webgl.log
```

```
# Windows (UNTESTED)
& "C:\Program Files\Unity\Hub\Editor\6000.6.6f1\Editor\Unity.exe" -batchmode -quit -projectPath
"C:\path\multiplayer game" -buildTarget WebGL -executeMethod BuildScript.BuildWebGL -logFile
build.log
```

macOS: `/Applications/Unity/Hub/Editor/6000.6.6f1/Unity.app/Contents/MacOS/Unity ...`. Quote project paths with spaces; `-logFile -` writes to stdout. Editor route in Unity 6: File, Build Profiles, add the Linux Dedicated Server profile (exact labels unverified), Switch Profile, Build. Server output: `MazeServer.x86_64` plus `MazeServer_Data/`; run `chmod +x`. Dedicated Server targets are optimized for CPU and memory, whereas plain `-batchmode -nographics` "doesn't perform any optimizations for running a build as a dedicated server" (Unity manual, summary), so use the Dedicated Server target rather than a normal Linux player.

Linux ARM: Unity Linux server builds are x86_64, so an ARM host (Oracle Ampere A1) cannot run them without emulation (general knowledge; verify Unity's current Linux ARM support).

4.4 Test the WebGL build locally

`file://` does not work (browsers block loading wasm; from knowledge). Unity's Build And Run starts a local server. By hand: `cd Builds/WebGL/<v> && python3 -m http.server 8000` works only for a Disabled-compression build or Decompression Fallback on, because Python sets no `Content-Encoding`. For Gzip/Brotli builds use the nginx config in Chapter 5 on the VM. Local test over `http://localhost` with `ws://` is fine for the first connection test (an http page can use `ws://`), but plain HTTP is not a secure context and does not prove the production behaviour, so repeat everything over HTTPS (Let's Encrypt on localhost: an HTTPS page still cannot call non-secure URLs).

Proof for Chapter 4: `ls Builds/LinuxServer/<v>/` shows the binary and data folder; running `./MazeServer.x86_64 -batchmode -nographics -port 7000 -roomId TEST1 -logFile -` prints `ROOM_READY TEST1 7000` and keeps running; the WebGL folder contains `Build/*.gz` files that load under the nginx config.

Chapter 5. The Linux server: nginx, Node lobby, systemd and room routing

Figure 3. Create and join a room

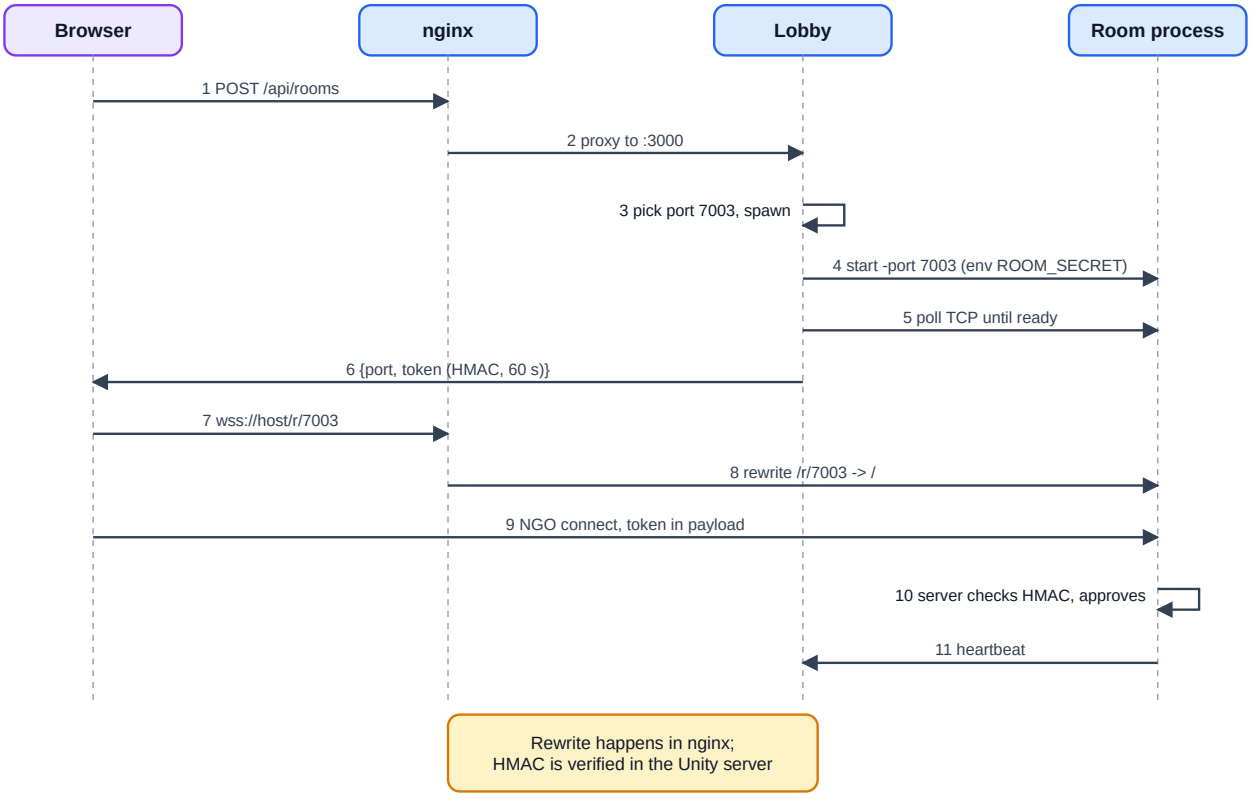


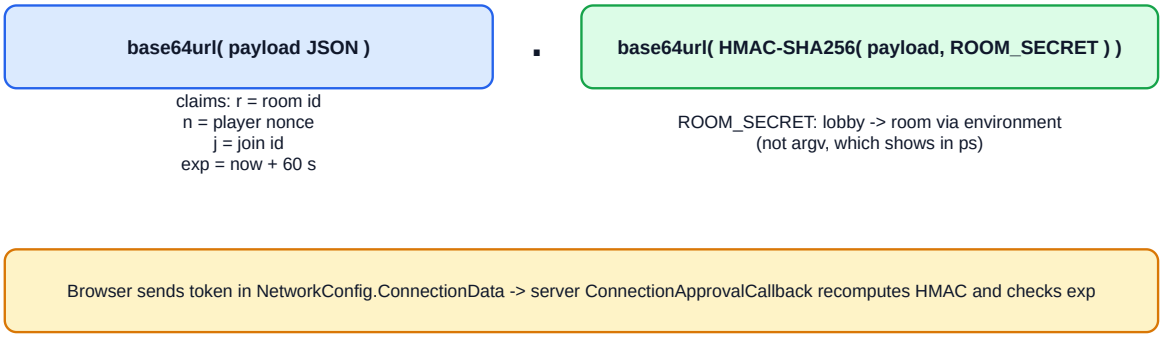
Figure 4. Room lifecycle



SIGTERM first, SIGKILL after 5 s. Port returns to the pool on process exit.

Orphan Unity servers are killed when the lobby starts; max-rooms cap rejects new rooms.

Figure 5. Join token anatomy



Pre-flight checklist

- [ ] VM from Chapter 2 with a trusted certificate in `/etc/letsencrypt/live/maze.duckdns.org/` (staging is acceptable while testing).
- [ ] `sudo apt install -y nginx rsync` done; nginx is 1.25.1 or newer for `http2 on;` (older: `listen 443 ssl http2;`).
- [ ] Node 22 LTS installed (the lobby was tested on v22.22.0). One way: NodeSource or `nvm`; the guide does not prescribe one.
- [ ] Service account `maze` exists (Chapter 2.8).
- [ ] Directories created: `sudo mkdir -p /opt/maze/{web,server,lobby} /etc/maze`.
- [ ] Secrets generated: `openssl rand -hex 32` (token secret), `openssl rand -hex 16` (internal key).
- [ ] You know the public host name, port (443) and that room ports 7000-7099 and lobby 3000 stay on loopback.

5.1 File layout

```
/opt/maze/web/      WebGL build (index.html, Build/*.gz|*.br, TemplateData/)  root:www-data
0755/0644
/opt/maze/server/   Linux server build (MazeServer.x86_64 + _Data/)            maze:maze 0755,
binary +x
/opt/maze/lobby/    package.json, src/, node_modules/                          maze:maze
/etc/maze/lobby.env secrets and config                                         root:maze 0640
/run/maze-lobby/    pid files (systemd RuntimeDirectory)
```

5.2 Request flow in words

1. The browser loads `https://HOST/` and the build files from nginx.
2. The game POSTs `/api/rooms` (create) or `/api/rooms/ABCDE/join`; nginx proxies to the lobby on 127.0.0.1:3000.
3. On create, the lobby allocates a free loopback port (7000-7099), spawns `MazeServer.x86_64 -port P -roomId ID -maxPlayers N`, waits until the TCP port accepts connections, then returns `{url, host, port: 443, secure: true, path: "/r/P", token}`.
4. The client sets host, port 443, TLS and `WebSocketPath = "/r/P"` and starts NGO with the token as approval payload.
5. nginx matches `^/r/(?<room_port>70[0-9][0-9])$`, rewrites the request URI to `/` and proxies the upgrade to `127.0.0.1:P`.

6. The room validates the HMAC token in connection approval and heartbeats to `/internal/rooms/ID/heartbeat` every 5 s; the lobby reaps rooms that stop reporting or stay empty past the grace period.

5.3 nginx configuration (UNTESTED)

nginx and Caddy were not installable in the research sandbox, so **no `nginx -t` has been run**. The WebSocket pattern (`proxy_http_version 1.1`, Upgrade/Connection from a `map`, raised `proxy_read_timeout`) matches the nginx WebSocket documentation as relayed by freenginx docs and websocket.org (summaries): nginx closes a proxied connection after 60 s without data by default, measured between reads, so pings also reset it. Two additions beyond the research notes are marked `# ADDED`: `gzip off` (a stock Ubuntu `nginx.conf` may enable gzip and re-compress already-compressed Unity files, producing the same parse error as missing headers) and the per-location header caveat. Run `sudo nginx -t` before every reload.

```
# /etc/nginx/sites-available/maze.conf    (UNTESTED)   ln -s into sites-enabled/
# http-context directives (sites-enabled is included inside http{})
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}
limit_conn_zone $binary_remote_addr zone=perip:10m;
limit_req_zone  $binary_remote_addr zone=api:10m rate=10r/s;

server {
    listen 80;
    listen [::]:80;
    server_name maze.duckdns.org;
    location /.well-known/acme-challenge/ { root /var/www/acme; }   # only used by HTTP-01 on a
public host
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;                       # nginx >= 1.25.1; older: "listen 443 ssl http2;"
    server_name maze.duckdns.org;

    ssl_certificate     /etc/letsencrypt/live/maze.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/maze.duckdns.org/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_session_cache shared:SSL:10m;

    server_tokens off;
    limit_conn perip 40;
    gzip off;                                           # ADDED: never re-compress Unity's pre-
compressed files
    add_header X-Content-Type-Options nosniff always;   # NOTE: any add_header inside a location
REPLACES inherited ones

    # --- Unity WebGL build (static) ---
    root /opt/maze/web;
    index index.html;

    location / {
        try_files $uri =404;
        add_header Cache-Control "no-cache";
        add_header X-Content-Type-Options nosniff always;   # repeated because this location sets
its own add_header
    }

    # Pre-compressed Unity files: correct Content-Encoding + real MIME type
    location ~ \.wasm\.br$   { types {} default_type application/wasm;         add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
```

```
    location ~ \.js\.br$     { types {} default_type application/javascript;   add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.data\.br$   { types {} default_type application/octet-stream; add_header Content-
Encoding br;   add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.wasm\.gz$   { types {} default_type application/wasm;         add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.js\.gz$     { types {} default_type application/javascript;   add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.data\.gz$   { types {} default_type application/octet-stream; add_header Content-
Encoding gzip; add_header Cache-Control "public, max-age=31536000, immutable"; }
    location ~ \.wasm$       { types {} default_type application/wasm; }

    # --- Lobby REST API ---
    location /api/ {
        limit_req zone=api burst=20 nodelay;
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;      # overwrite, never append a client-
supplied value
        proxy_set_header X-Forwarded-Proto $scheme;
        client_max_body_size 4k;
    }
    location /internal/ { return 404; }                     # heartbeat endpoint is loopback only

    # --- Game rooms: wss://HOST/r/7000 .. /r/7099 -> ws://127.0.0.1:<port>/ ---
    location ~ ^/r/(?<room_port>70[0-9][0-9])$ {
        proxy_pass http://127.0.0.1:$room_port/;            # variable + URI: the URI "/" replaces
/r/70xx; IP literal needs no resolver
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_read_timeout 3600s;                           # default 60 s would cut idle sockets
        proxy_send_timeout 3600s;
        proxy_buffering off;
        limit_conn perip 12;
        client_max_body_size 0;
    }
}
```

Caveats that need a real run. (1) With a variable in `proxy_pass` plus an explicit URI, nginx sends that URI as written instead of the request URI (my recollection of the nginx docs, not re-read); verify with the room server's access log or the spike. (2) If the named-capture regex form is rejected by your nginx, use a generated `map $uri $room_upstream { include /etc/nginx/maze-rooms.map; }` plus a reload. (3) A dead room produces `502` to the client. (4) Regex locations beat the plain `/api/` prefix only if the URI also matches the regex; `/api/...` never ends in `.wasm`, so there is no conflict. (5) On a rejected or unusual `Origin`, add the origin check described in Chapter 9. The 60-second idle timeout rationale: websocket.org.

5.4 Caddy alternative (UNTESTED)

Caddy's `reverse_proxy` proxies WebSockets automatically; `stream_close_delay` keeps long streams alive across config reloads, which can otherwise close WebSockets (Caddy docs, community, summaries). The `{re.room.1}` placeholder used after a `rewrite` was not confirmed; if it fails, capture the port with a `map` first.

```
{
    email you@example.com
}
maze.duckdns.org {
    tls { dns duckdns {env.DUCKDNS_API_TOKEN} }      # needs the xcaddy build from Chapter 2.5; on
a public name drop this block
```

```
    @room path_regexp room ^/r/(70[0-9][0-9])$
    handle @room {
        rewrite * /
        reverse_proxy 127.0.0.1:{re.room.1} {
            stream_close_delay 5m
        }
    }
    handle /api/* { reverse_proxy 127.0.0.1:3000 }
    respond /internal/* 404
    handle {
        root * /opt/maze/web
        @br path *.br
        @gz path *.gz
        header @br Content-Encoding br
        header @gz Content-Encoding gzip
        @wasm path *.wasm.br *.wasm.gz *.wasm
        header @wasm Content-Type application/wasm
        file_server
    }
}
```

Do not put `encode zstd gzip` on Unity's `.br` / `.gz` files. Caddy may label `.gz` files `application/gzip`, in which case use `header @wasm >Content-Type application/wasm` (flag for testing).

Concern	nginx	Caddy
Dynamic rooms	Port from URL regex, no reload	Port from placeholder, no reload
Reload effect on live sockets	Old workers keep serving until connections close	May drop WebSockets unless `stream_close_delay`
TLS	certbot/acme.sh	Automatic; DuckDNS needs custom build
Rate limiting	Built in (`limit_req`, `limit_conn`)	Third-party module `caddy-ratelimit` (repo mirror)
Raw TCP/SNI routing	`stream` module	`caddy-l4` plugin

5.5 The Node lobby (TESTED)

Tested on Node 22.22.0 with a stand-in room server: create (201 with token and `/r/<port>` path), list, join, validation rejecting bad names, empty-room reaper stopping the child, SIGTERM killing children. Not tested with the real Unity binary. Package versions were queried live from the npm registry on 2026-10-26: fastify 5.12.7, @fastify/cors 11.3.1, @fastify/rate-limit 11.2.1, ws 8.22.0 (npm registry). Tokens use Node's built-in `crypto`, so `jsonwebtoken` is not needed. Known limits: room state is in memory (a lobby restart ends all rooms), player counts come only from heartbeats, and `DELETE` accepts any valid token for the room (add an owner flag if needed).

/opt/maze/lobby/package.json (TESTED)

```
{
  "name": "maze-lobby",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "engines": { "node": ">=22" },
  "scripts": { "start": "node src/server.mjs" },
  "dependencies": {
    "@fastify/cors": "11.3.1",
    "@fastify/rate-limit": "11.2.1",
    "fastify": "5.12.7"
  },
  "devDependencies": {
```

```
    "ws": "8.22.0"
  }
}
```

src/config.mjs (TESTED)

```
import process from 'node:process';

const int = (k, d) => (process.env[k] ? parseInt(process.env[k], 10) : d);

export const cfg = {
  host: process.env.LOBBY_HOST ?? '127.0.0.1',
  port: int('LOBBY_PORT', 3000),
  serverBin: process.env.SERVER_BIN ?? '/opt/maze/server/MazeServer.x86_64',
  serverCwd: process.env.SERVER_CWD ?? '/opt/maze/server',
  portMin: int('PORT_MIN', 7000),
  portMax: int('PORT_MAX', 7099),
  maxRooms: int('MAX_ROOMS', 8),
  maxPlayers: int('MAX_PLAYERS', 8),
  // secret shared with the Unity server (env var, never argv: argv is visible in `ps`)
  tokenSecret: process.env.TOKEN_SECRET ?? '',
  internalKey: process.env.INTERNAL_KEY ?? '',
  tokenTtlSec: int('TOKEN_TTL_SEC', 60),
  readyTimeoutMs: int('READY_TIMEOUT_MS', 30000),
  emptyGraceMs: int('EMPTY_GRACE_MS', 60000),
  heartbeatTimeoutMs: int('HEARTBEAT_TIMEOUT_MS', 30000),
  // what the browser must use to reach a room through nginx
  publicHost: process.env.PUBLIC_HOST ?? 'localhost',
  publicPort: int('PUBLIC_PORT', 443),
  publicTls: (process.env.PUBLIC_TLS ?? 'true') === 'true',
  pathPrefix: process.env.ROOM_PATH_PREFIX ?? '/r/',
  runDir: process.env.RUN_DIR ?? '/run/maze-lobby',
  allowedOrigin: process.env.ALLOWED_ORIGIN ?? '', // empty = same-origin only, no CORS headers
};

if (cfg.tokenSecret.length < 32 || cfg.internalKey.length < 16) {
  console.error('TOKEN_SECRET (>=32 chars) and INTERNAL_KEY (>=16 chars) are required');
  process.exit(1);
}
```

src/tokens.mjs (TESTED)

```
import { createHmac, randomBytes, timingSafeEqual } from 'node:crypto';
import { cfg } from './config.mjs';

const b64u = (b) => Buffer.from(b).toString('base64url');
const sign = (data) => createHmac('sha256', cfg.tokenSecret).update(data).digest();

// token = base64url(JSON payload) + "." + base64url(HMAC-SHA256(secret, base64url(payload)))
export function makeJoinToken(roomId, name) {
  const payload = b64u(JSON.stringify({
    r: roomId,
    n: name,
    j: randomBytes(8).toString('hex'),
    exp: Math.floor(Date.now() / 1000) + cfg.tokenTtlSec,
  }));
  return `${payload}.${b64u(sign(payload))}`;
}

// The Unity server does the same check in ConnectionApproval; this is used by tests.
export function verifyJoinToken(token, roomId) {
  const [payload, mac] = String(token).split('.');
  if (!payload || !mac) return null;
  const good = sign(payload);
  const got = Buffer.from(mac, 'base64url');
  if (got.length !== good.length || !timingSafeEqual(got, good)) return null;
  const p = JSON.parse(Buffer.from(payload, 'base64url').toString());
  if (p.r !== roomId || p.exp < Date.now() / 1000) return null;
```

```
  return p;
}
```

src/rooms.mjs (TESTED, with a stand-in room server)

```
import { spawn } from 'node:child_process';
import { randomInt } from 'node:crypto';
import net from 'node:net';
import fs from 'node:fs';
import path from 'node:path';
import { cfg } from './config.mjs';

const ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'; // no 0/O/1/I
export const rooms = new Map(); // id -> room
const usedPorts = new Set();

export function newRoomId() {
  for (let i = 0; i < 50; i++) {
    let id = '';
    for (let k = 0; k < 5; k++) id += ALPHABET[randomInt(ALPHABET.length)];
    if (!rooms.has(id)) return id;
  }
  throw new Error('could not allocate room id');
}

function canBind(port) {
  return new Promise((resolve) => {
    const s = net.createServer();
    s.once('error', () => resolve(false));
    s.listen(port, '127.0.0.1', () => s.close(() => resolve(true)));
  });
}

export async function allocPort() {
  for (let p = cfg.portMin; p <= cfg.portMax; p++) {
    if (usedPorts.has(p)) continue;
    usedPorts.add(p); // reserve first: avoids a race between concurrent creates
    if (await canBind(p)) return p;
    usedPorts.delete(p);
  }
  return null;
}

function waitTcp(port, timeoutMs, aborted) {
  const t0 = Date.now();
  return new Promise((resolve) => {
    const tick = () => {
      if (aborted()) return resolve(false);
      const s = net.connect(port, '127.0.0.1');
      s.once('connect', () => { s.destroy(); resolve(true); });
      s.once('error', () => {
        s.destroy();
        if (Date.now() - t0 > timeoutMs) resolve(false);
        else setTimeout(tick, 250);
      });
    };
    tick();
  });
}

const pidFile = (id) => path.join(cfg.runDir, `${id}.pid`);

export async function createRoom({ name, maxPlayers }, log) {
  if (rooms.size >= cfg.maxRooms) return { error: 'server_full' };
  const port = await allocPort();
  if (port === null) return { error: 'no_free_port' };
  const id = newRoomId();
  const room = {
    id, port, name, maxPlayers: Math.min(maxPlayers ?? cfg.maxPlayers, cfg.maxPlayers),
    state: 'starting', players: 0, createdAt: Date.now(),
    lastHeartbeat: Date.now(), emptySince: Date.now(), child: null, exited: false,
```

```
  };
  rooms.set(id, room);

  const args = ['-batchmode', '-nographics', '-port', String(port), '-roomId', id,
    '-maxPlayers', String(room.maxPlayers), '-logFile', '-']; // '-' = stdout -> journald
  const child = spawn(cfg.serverBin, args, {
    cwd: cfg.serverCwd,
    stdio: ['ignore', 'pipe', 'pipe'],
    env: {
      // minimal env; secrets go here, not in argv
      PATH: process.env.PATH,
      HOME: cfg.serverCwd,
      ROOM_SECRET: cfg.tokenSecret,
      LOBBY_URL: `http://127.0.0.1:${cfg.port}`,
      LOBBY_INTERNAL_KEY: cfg.internalKey,
    },
  });
  room.child = child;
  try { fs.writeFileSync(pidFile(id), String(child.pid)); } catch { /* ignore */ }

  const pipeLines = (stream, level) => {
    let buf = '';
    stream.on('data', (d) => {
      buf += d;
      let i;
      while ((i = buf.indexOf('\n')) >= 0) {
        const line = buf.slice(0, i); buf = buf.slice(i + 1);
        if (line.includes('ROOM_READY')) room.markerSeen = true;
        log[level]({ room: id, port }, line);
      }
    });
  };
  pipeLines(child.stdout, 'info');
  pipeLines(child.stderr, 'warn');

  child.once('exit', (code, sig) => {
    room.exited = true;
    log.info({ room: id, code, sig }, 'room exited');
    destroyRecord(id);
  });
  child.once('error', (e) => { log.error({ room: id, err: e.message }, 'spawn failed'); room.exited
= true; destroyRecord(id); });

  const ok = await waitTcp(port, cfg.readyTimeoutMs, () => room.exited);
  if (!ok) { killRoom(id, 'not_ready'); return { error: 'room_failed_to_start' }; }
  room.state = 'ready';
  room.lastHeartbeat = Date.now();
  return { room };
}

function destroyRecord(id) {
  const r = rooms.get(id);
  if (!r) return;
  usedPorts.delete(r.port);
  rooms.delete(id);
  try { fs.unlinkSync(pidFile(id)); } catch { /* ignore */ }
}

export function killRoom(id, reason = 'api') {
  const r = rooms.get(id);
  if (!r) return false;
  r.state = 'closing';
  if (r.child && !r.exited) {
    r.child.kill('SIGTERM');
    const t = setTimeout(() => { if (!r.exited) r.child.kill('SIGKILL'); }, 5000);
    t.unref();
  } else destroyRecord(id);
  return reason;
}

export function reap(log) {
  const now = Date.now();
  for (const r of rooms.values()) {
    if (r.state !== 'ready') continue;
```

```
    if (now - r.lastHeartbeat > cfg.heartbeatTimeoutMs) { log.warn({ room: r.id }, 'reap: heartbeat
lost'); killRoom(r.id, 'heartbeat'); }
    else if (r.players === 0 && now - r.emptySince > cfg.emptyGraceMs) { log.info({ room: r.id },
'reap: empty'); killRoom(r.id, 'empty'); }
  }
}

export function heartbeat(id, players) {
  const r = rooms.get(id);
  if (!r) return false;
  r.lastHeartbeat = Date.now();
  if (players !== r.players) r.players = players;
  if (players > 0) r.emptySince = 0; else if (!r.emptySince) r.emptySince = Date.now();
  return true;
}

// Startup: kill leftovers from a previous lobby that crashed (pid files + /proc check).
export function cleanOrphans(log) {
  fs.mkdirSync(cfg.runDir, { recursive: true });
  for (const f of fs.readdirSync(cfg.runDir)) {
    if (!f.endsWith('.pid')) continue;
    const file = path.join(cfg.runDir, f);
    const pid = parseInt(fs.readFileSync(file, 'utf8'), 10);
    try {
      const cmd = fs.readFileSync(`/proc/${pid}/cmdline`, 'utf8');
      if (cmd.includes(path.basename(cfg.serverBin))) { process.kill(pid, 'SIGKILL'); log.warn({
pid }, 'killed orphan'); }
    } catch { /* process gone */ }
    try { fs.unlinkSync(file); } catch { /* ignore */ }
  }
}

export async function shutdownAll(log) {
  const live = [...rooms.values()].filter((r) => r.child && !r.exited);
  live.forEach((r) => r.child.kill('SIGTERM'));
  const deadline = Date.now() + 5000;
  while (live.some((r) => !r.exited) && Date.now() < deadline) await new Promise((r) =>
setTimeout(r, 100));
  live.filter((r) => !r.exited).forEach((r) => { log.warn({ room: r.id }, 'SIGKILL at shutdown');
r.child.kill('SIGKILL'); });
}
```

src/server.mjs (TESTED)

```
import Fastify from 'fastify';
import cors from '@fastify/cors';
import rateLimit from '@fastify/rate-limit';
import { timingSafeEqual } from 'node:crypto';
import { cfg } from './config.mjs';
import { makeJoinToken } from './tokens.mjs';
import { rooms, createRoom, killRoom, reap, heartbeat, cleanOrphans, shutdownAll } from
'./rooms.mjs';

const app = Fastify({
  logger: { level: process.env.LOG_LEVEL ?? 'info' },
  trustProxy: '127.0.0.1', // nginx on the same box sets X-Forwarded-For
  bodyLimit: 4096,
});

if (cfg.allowedOrigin) await app.register(cors, { origin: cfg.allowedOrigin, methods: ['GET',
'POST', 'DELETE'] });
await app.register(rateLimit, { global: true, max: 60, timeWindow: '1 minute' });

const ROOM_ID = { type: 'string', pattern: '^[A-Z0-9]{5}$' };
const NAME = { type: 'string', minLength: 1, maxLength: 24, pattern: '^[\\w .-]+$' };

const publicRoom = (r) => ({ id: r.id, name: r.name, players: r.players, maxPlayers: r.maxPlayers,
state: r.state });
const connInfo = (r, token) => ({
  url: `${cfg.publicTls ? 'wss' :
'ws'}://${cfg.publicHost}:${cfg.publicPort}${cfg.pathPrefix}${r.port}`,
```

```
  host: cfg.publicHost, port: cfg.publicPort, secure: cfg.publicTls,
  path: `${cfg.pathPrefix}${r.port}`, token,
});

app.get('/api/health', async () => ({ ok: true, rooms: rooms.size }));

app.get('/api/rooms', async () => ({
  rooms: [...rooms.values()].filter((r) => r.state === 'ready').map(publicRoom),
}));

app.post('/api/rooms', {
  config: { rateLimit: { max: 5, timeWindow: '1 minute' } },
  schema: { body: { type: 'object', additionalProperties: false, required: ['playerName'],
    properties: { playerName: NAME, roomName: NAME, maxPlayers: { type: 'integer', minimum: 1,
maximum: 16 } } } },
}, async (req, reply) => {
  const { playerName, roomName, maxPlayers } = req.body;
  const res = await createRoom({ name: roomName ?? `${playerName}'s room`, maxPlayers }, req.log);
  if (res.error) return reply.code(res.error === 'room_failed_to_start' ? 500 : 503).send({ error:
res.error });
  return reply.code(201).send({ room: publicRoom(res.room), ...connInfo(res.room,
makeJoinToken(res.room.id, playerName)) });
});

app.post('/api/rooms/:id/join', {
  config: { rateLimit: { max: 20, timeWindow: '1 minute' } },
  schema: { params: { type: 'object', properties: { id: ROOM_ID } },
    body: { type: 'object', additionalProperties: false, required: ['playerName'], properties: {
playerName: NAME } } },
}, async (req, reply) => {
  const r = rooms.get(req.params.id.toUpperCase());
  if (!r || r.state !== 'ready') return reply.code(404).send({ error: 'no_such_room' });
  if (r.players >= r.maxPlayers) return reply.code(409).send({ error: 'room_full' });
  return { room: publicRoom(r), ...connInfo(r, makeJoinToken(r.id, req.body.playerName)) };
});

// Only the room owner / admin should delete. Here: requires a valid join token for that room.
app.delete('/api/rooms/:id', { schema: { params: { type: 'object', properties: { id: ROOM_ID } } }
}, async (req, reply) => {
  const auth = String(req.headers.authorization ?? '').replace(/^Bearer /, '');
  const { verifyJoinToken } = await import('./tokens.mjs');
  if (!verifyJoinToken(auth, req.params.id)) return reply.code(403).send({ error: 'forbidden' });
  return killRoom(req.params.id) ? { ok: true } : reply.code(404).send({ error: 'no_such_room' });
});

// Called by the Unity server every ~5 s over loopback. nginx must NOT proxy /internal/.
app.post('/internal/rooms/:id/heartbeat', {
  config: { rateLimit: false },
  schema: { params: { type: 'object', properties: { id: ROOM_ID } },
    body: { type: 'object', required: ['players'], properties: { players: { type: 'integer',
minimum: 0, maximum: 64 } } } },
}, async (req, reply) => {
  const k = Buffer.from(String(req.headers['x-internal-key'] ?? ''));
  const want = Buffer.from(cfg.internalKey);
  if (k.length !== want.length || !timingSafeEqual(k, want)) return reply.code(403).send();
  return heartbeat(req.params.id, req.body.players) ? { ok: true } : reply.code(404).send();
});

cleanOrphans(app.log);
const reaper = setInterval(() => reap(app.log), 5000);
reaper.unref();

let stopping = false;
async function stop(sig) {
  if (stopping) return;
  stopping = true;
  app.log.info({ sig }, 'shutting down');
  clearInterval(reaper);
  await app.close();
  await shutdownAll(app.log);
  process.exit(0);
}
process.on('SIGTERM', () => stop('SIGTERM'));
process.on('SIGINT', () => stop('SIGINT'));
```

```
await app.listen({ host: cfg.host, port: cfg.port });
```

tools/fake-room-server.mjs (TESTED; stand-in for the Unity binary, shows the heartbeat contract)

```
// Accepts the same CLI args, listens (plain ws) on 127.0.0.1:<port>, only on path "/",
// POSTs heartbeats to the lobby like the real server should.
import { WebSocketServer } from 'ws';
const a = process.argv; const arg = (n) => a[a.indexOf(n) + 1];
const port = +arg('-port'), roomId = arg('-roomId');
const wss = new WebSocketServer({ host: '127.0.0.1', port, path: '/' });
let players = 0;
wss.on('connection', (ws) => { players++; ws.on('message', (m, bin) => ws.send(m, { binary: bin
})); ws.on('close', () => players--); });
console.log(`ROOM_READY ${roomId} ${port}`);
setInterval(() => fetch(`${process.env.LOBBY_URL}/internal/rooms/${roomId}/heartbeat`, {
  method: 'POST', headers: { 'content-type': 'application/json', 'x-internal-key':
process.env.LOBBY_INTERNAL_KEY },
  body: JSON.stringify({ players }) }).catch(() => {}), 2000);
process.on('SIGTERM', () => process.exit(0));
```

To test the lobby without Unity, point `SERVER_BIN` at a small wrapper script that runs `node tools/fake-room-server.mjs "$@"` (the lobby spawns the binary directly with the Unity-style arguments), set `PORT_MIN` / `PORT_MAX`, and use `curl -s -X POST localhost:3000/api/rooms -H 'content-type: application/json' -d '{"playerName":"Ann"}'`.

5.6 Configuration file and systemd (UNTESTED)

systemd directives follow standard `systemd.exec` and `systemd.kill` semantics; no systemd existed in the sandbox. Children share the unit's cgroup, so `KillMode=control-group` stops all rooms with the lobby. Hardening options are standard (OneUptime hardening guide, fastdeploy docs, summaries); audit the result with `systemd-analyze security maze-lobby`.

```
# /etc/systemd/system/maze-lobby.service   (UNTESTED)
[Unit]
Description=Maze lobby and room supervisor
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=maze
Group=maze
WorkingDirectory=/opt/maze/lobby
EnvironmentFile=/etc/maze/lobby.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node src/server.mjs
Restart=on-failure
RestartSec=2

# children (Unity servers) live in this unit's cgroup and die with it
KillMode=control-group
KillSignal=SIGTERM
TimeoutStopSec=15
RuntimeDirectory=maze-lobby
RuntimeDirectoryMode=0750

LimitNOFILE=8192
TasksMax=512
MemoryMax=1700M
MemoryHigh=1500M

NoNewPrivileges=true
ProtectSystem=strict
```

```
ProtectHome=true
PrivateTmp=true
PrivateDevices=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX
RestrictNamespaces=true
LockPersonality=true
SystemCallArchitectures=native
ReadWritePaths=/opt/maze/server

StandardOutput=journal
StandardError=journal
SyslogIdentifier=maze-lobby

[Install]
WantedBy=multi-user.target
```

If Unity fails to start under `ProtectSystem=strict`, it probably needs a writable home or cache (`~/.config/unity3d`); the lobby sets `HOME` to the server directory, which is in `ReadWritePaths`. Loosen options one at a time while reading `journalctl -u maze-lobby`.

```
# /etc/maze/lobby.env   (chmod 640, owner root:maze)
LOBBY_PORT=3000
SERVER_BIN=/opt/maze/server/MazeServer.x86_64
SERVER_CWD=/opt/maze/server
PORT_MIN=7000
PORT_MAX=7099
MAX_ROOMS=8
PUBLIC_HOST=maze.duckdns.org
PUBLIC_PORT=443
PUBLIC_TLS=true
TOKEN_SECRET=<openssl rand -hex 32>
INTERNAL_KEY=<openssl rand -hex 16>
```

```
# /etc/systemd/journald.conf.d/maze.conf
[Journal]
SystemMaxUse=200M
MaxRetentionSec=1week
```

Enable: `sudo systemctl daemon-reload && sudo systemctl enable --now maze-lobby`. nginx logs rotate through the stock logrotate file. Use `-logFile -` for Unity so logs go to the journal rather than an unbounded file (flag assumed from Unity's documented behaviour; not tested).

5.7 Deploy script (UNTESTED)

```
#!/usr/bin/env bash
# deploy.sh -- run from the dev box:  ./deploy.sh user@host
set -euo pipefail
HOST=${1:?user@host}
rsync -az --delete WebGL-build/       "$HOST:/tmp/maze-web/"
rsync -az --delete LinuxServer-build/ "$HOST:/tmp/maze-server/"
rsync -az --delete --exclude node_modules lobby/ "$HOST:/tmp/maze-lobby/"
ssh "$HOST" 'set -e
  sudo rsync -a --delete /tmp/maze-web/    /opt/maze/web/
  sudo rsync -a --delete /tmp/maze-server/ /opt/maze/server/
  sudo rsync -a --delete --exclude node_modules /tmp/maze-lobby/ /opt/maze/lobby/
  sudo chown -R maze:maze /opt/maze/server /opt/maze/lobby
  sudo chown -R root:www-data /opt/maze/web && sudo find /opt/maze/web -type d -exec chmod 755 {} +
&& sudo find /opt/maze/web -type f -exec chmod 644 {} +
  sudo chmod +x /opt/maze/server/MazeServer.x86_64
  (cd /opt/maze/lobby && sudo -u maze npm ci --omit=dev)
```

```
  sudo nginx -t && sudo systemctl reload nginx
  sudo systemctl restart maze-lobby'
```

Restarting the lobby kills running rooms; acceptable for a prototype, otherwise drain first. `npm ci` needs a `package-lock.json` committed in `lobby/` (generate with `npm install`). Rollback is redeploying the previous build folder.

5.8 Sizing

No measured numbers exist. Reports of headless Linux Unity memory are anecdotal: about 130 MB for a very simple game, 800 MB for a heavier one (Unity forum "Headless server", summary); a tracked Unity issue reports Linux headless CPU usage rising over time (Issue Tracker, summary). Planning figure (estimate, not measured): 100-250 MB RSS per small room with the Dedicated Server target, so 6-12 rooms fit in 2 GB after about 400 MB for OS, nginx and lobby; hence `MAX_ROOMS=8` and `MemoryMax=1700M`. Measure with `ps -o rss= -p <pid>` after two players join and set `MAX_ROOMS` from the result. Set `Application.targetFrameRate` (30-60); a spinning batchmode loop is the main CPU risk on 1-2 vCPUs.

5.9 Fallback routes if the spike fails

Plan B, public port range (generator TESTED for output shape only). One TLS listener per port, so the client uses `Port = 8000+n` and path `/`; needs ufw `8000:8099/tcp`, a VirtualBox forward per port, and fails on networks that allow only 443.

```
// tools/gen-port-range.mjs   usage: node gen-port-range.mjs 8000 8099 7000 >
/etc/nginx/conf.d/maze-ports.conf
const [pubStart, pubEnd, privStart] = process.argv.slice(2).map(Number);
let out = '# generated; one TLS listener per public port\n';
for (let p = pubStart; p <= pubEnd; p++) {
  const t = privStart + (p - pubStart);
  out += `server {
  listen ${p} ssl;
  server_name _;
  ssl_certificate     /etc/letsencrypt/live/maze.duckdns.org/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/maze.duckdns.org/privkey.pem;
  location / {
    proxy_pass http://127.0.0.1:${t};
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $connection_upgrade;
    proxy_read_timeout 3600s; proxy_send_timeout 3600s;
  }
}
`;
}
process.stdout.write(out);
```

(The original draft used `/etc/letsencrypt/live/maze/`; the path above matches the certificate name used in this guide.)

Plan C, Node upgrade router (TESTED: echoed a binary frame through `/r/7003`, returned 404 for `/r/9999`). nginx proxies `/ws` or `/r/` to 127.0.0.1:7999; the router parses only the HTTP upgrade request, rewrites the target to `/`, then pipes raw bytes, so Unity frames pass through unchanged. It also accepts a `room=NNNN` cookie. Costs one Node hop per player.

```
// tools/ws-router.mjs
import http from 'node:http';
import net from 'node:net';

const PORT_MIN = 7000, PORT_MAX = 7099;
const pick = (req) => {
  const m = /^\/r\/(\d{4})$/.exec(req.url.split('?')[0]);
  const c = /(?:^|;\s*)room=(\d{4})/.exec(req.headers.cookie ?? '');
  const p = +(m?.[1] ?? c?.[1] ?? 0);
  return p >= PORT_MIN && p <= PORT_MAX ? p : 0;
};

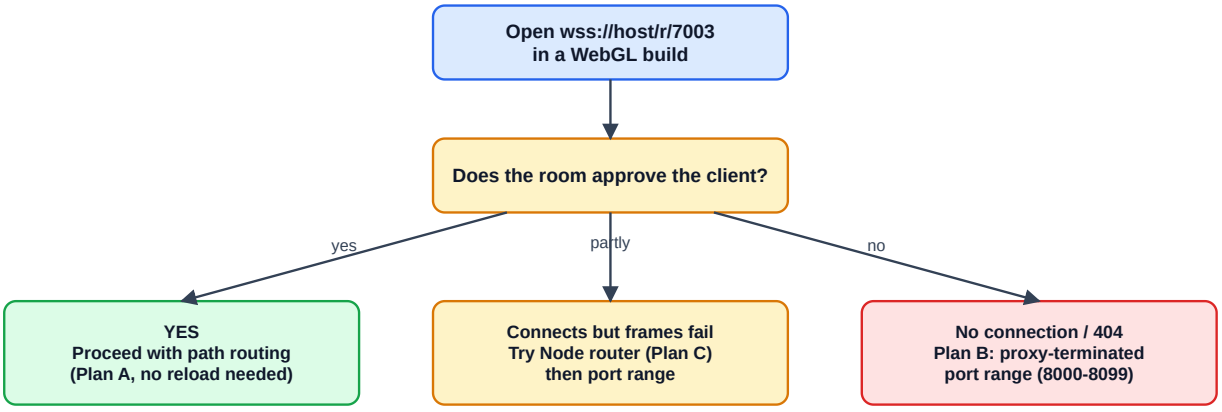
const srv = http.createServer((_, res) => { res.writeHead(426).end('upgrade required'); });
srv.on('upgrade', (req, client, head) => {
  const port = pick(req);
  if (!port) return client.end('HTTP/1.1 404 Not Found\r\nConnection: close\r\n\r\n');
  const up = net.connect(port, '127.0.0.1', () => {
    let raw = `GET / HTTP/1.1\r\n`;
    for (let i = 0; i < req.rawHeaders.length; i += 2) raw += `${req.rawHeaders[i]}:
${req.rawHeaders[i + 1]}\r\n`;
    up.write(raw + '\r\n');
    if (head?.length) up.write(head);
    client.pipe(up).pipe(client);
  });
  const kill = () => { client.destroy(); up.destroy(); };
  up.on('error', kill); client.on('error', kill); up.on('close', kill); client.on('close', kill);
});
srv.listen(7999, '127.0.0.1');
```

Other options, ranked lower: a subdomain per room needs a wildcard certificate and DuckDNS holds a single TXT (fragile; rejected); a cookie set by the join call and routed with `map $cookie_room` works without Unity cooperation but allows only one room per browser profile (UNTESTED); `Sec-WebSocket-Protocol` cannot be controlled from Unity's plugin; hosting many NGO sessions in one process is unsupported as far as found.

Proof for Chapter 5: `sudo nginx -t` passes; `curl -I https://maze.duckdns.org/Build/<file>.wasm.gz` shows `content-type: application/wasm` and `content-encoding: gzip`; `curl -s -X POST https://maze.duckdns.org/api/rooms -H 'content-type: application/json' -d '{"playerName":"Ann"}'` returns 201 with a `path` like `/r/7000`; `journalctl -u maze-lobby` shows `ROOM_READY`.

Chapter 6. The 10-minute spike that proves path routing

Figure 9. Routing spike decision tree



Unproven: WebGL plugin sends the path; TLS proxy is transparent to Unity Transport frames.

This chapter exists because the whole design hangs on three claims no one has run: the WebGL plugin sends `WebSocketPath`, a TLS-terminating proxy is transparent to Unity Transport, and `UseEncryption = true` on the client means `wss`. **Do this before writing game logic.** Everything here is UNTESTED.

Pre-flight checklist

- [ ] Chapter 2 done: a trusted (staging is not trusted by browsers; use production or a locally trusted CA) certificate and nginx serving HTTPS.
- [ ] A minimal Unity project with NGO 2.13.3, one scene with `NetworkManager` + `UnityTransport` (Use Web Sockets on), a trivial player prefab, `ServerBootstrap` and `ClientConnect` from Chapter 3.
- [ ] One Linux server build in `/opt/maze/server`, the WebGL build in `/opt/maze/web`.

6.1 Steps

1. Log the incoming URI so success is visible. In the `http{}` context add, and in the `/r/` location add `access_log /var/log/nginx/rooms.log rooms;`:

```
log_format rooms '$remote_addr [$time_local] "$request_method $request_uri" $status
upstream=$upstream_addr up_status=$upstream_status';
```

2. Prove the proxy layer with a stand-in (no Unity needed). Run `node tools/fake-room-server.mjs -port 7000 -roomId SPIKE` with `LOBBY_URL` unset (heartbeat errors are swallowed), then:

```
curl -i -N --http1.1 -H "Connection: Upgrade" -H "Upgrade: websocket" -H "Sec-WebSocket-Version:
13" \
  -H "Sec-WebSocket-Key: SGVsbG8sIHdvcmxkIQ==" -H "Origin: https://maze.duckdns.org"
https://maze.duckdns.org/r/7000
```

Expect `HTTP/1.1 101 Switching Protocols`, and `GET /r/7000` in `rooms.log`. 502 means the room is down; 404 means a wrong path or regex; 400/426 from upstream still proves forwarding. If the stand-in's `path: '/'` rejects the connection, the rewrite did not happen: fix the `proxy_pass` first.

3. Prove the Unity server accepts the rewritten request. Start the real server: `./MazeServer.x86_64 -batchmode -nographics -port 7000 -roomId SPIKE -logFile -` with `ROOM_SECRET` set to a 32+ character string.
4. Prove the WebGL client. In the page, call `Join("maze.duckdns.org", 443, "/r/7000", true, token)` with a token produced by the lobby (or temporarily disable approval for the spike). Open DevTools, Network, WS.
5. Record outcomes in this matrix:

Observation	Meaning
`rooms.log` shows `GET /r/7000` and status 101	Plugin sends the path and `wss` works. Claim 1 and 3 proven.
101, but NGO never connects or the server logs nothing	Frames dropped or protocol mismatch. Compare with PlayFab issue #9 symptoms; test with the server on a direct TLS listener (`SetServerSecrets`) to isolate the proxy.
`rooms.log` shows `GET /`	Plugin ignored the path (or `SetConnectionData` reset it). Re-check ordering; if still `/`, switch to Plan B or C.
Browser: mixed content error	`UseEncryption` did not select `wss`; try `SetClientSecrets` placement, or investigate `WithSecureClientParameters` mapping
Disconnect after about 60 s idle	`proxy_read_timeout` not applied to this location
Works, but disconnects after minutes in a hidden tab	Expected throttling; Chapter 3.9

6. Check the failure path: kill the room and confirm the client reports a clean failure rather than hanging; connect with an expired token and confirm `DisconnectReason`.

Decision rule: if step 4 shows `GET /r/7000` with 101 and an NGO connection completes, adopt this guide as written. If not, adopt Plan B (port range) or Plan C (router), which keep the rest of the guide intact.

Chapter 7. Testing strategy

Pre-flight checklist

- [ ] Chapter 6 spike passed (or fallback chosen).
- [ ] Two separate browser profiles or one normal plus one incognito window, placed side by side (not stacked tabs).
- [ ] `openssl`, `curl`, `websocat` available on the test machine.

7.1 Test ladder

Rung	Tool	What it proves
1	Multiplayer Play Mode (Unity 6+, Windows/macOS; up to 4 editor players, plus up to 4 locally built instances in 2.0 which needs Editor 6000.3.0b4 or later) (MPPM 2.0 manual, summary)	Netcode logic, RPCs, NetworkVariables; not WebGL behaviour
2	Editor-hosted server + WebGL build in two windows over `http://localhost` and `ws://`	WebGL transport works at all
3	Full path on the VM through nginx with real TLS	Everything in this guide
4	Hidden-tab and kill-server drills	Resilience (Chapter 3.9)

7.2 Why a Node `ws` bot cannot play

UTP WebSockets only interoperate with UTP, so a Node bot can test the TLS plus HTTP 101 layer (the proxy completes the upgrade before UTP sees anything) and connection capacity, but cannot send valid NGO frames. The lobby REST/WS layer can be fully tested with Node. For protocol-level load, run headless Unity clients (`-batchmode -nographics`). I found no project showing a Node client speaking UTP.

7.3 Smoke tests (UNTESTED, standard tooling)

```
# TLS and expiry
echo | openssl s_client -connect maze.duckdns.org:443 -servername maze.duckdns.org 2>/dev/null |
openssl x509 -noout -dates -subject -issuer
openssl s_client -connect maze.duckdns.org:443 -servername maze.duckdns.org -tls1_3 </dev/null |
head -5   # must work; -tls1_1 must fail
# static assets: headers must be right
curl -sI https://maze.duckdns.org/Build/<name>.wasm.gz | grep -i -E 'content-type|content-encoding'
# lobby
curl -fsS https://maze.duckdns.org/api/health
curl -s -X POST https://maze.duckdns.org/api/rooms -H 'content-type: application/json' -d
'{"playerName":"Ann"}'
curl -s https://maze.duckdns.org/api/rooms
# WebSocket upgrade (see Chapter 6 step 2); wrong Origin must be rejected once Chapter 9 is applied
# internal endpoint must not be public
curl -s -o /dev/null -w '%{http_code}\n' -X POST
https://maze.duckdns.org/internal/rooms/AAAAA/heartbeat   # expect 404
```

Load testing: for the lobby use `autocannon`, `k6` or `artillery` (free; general knowledge). For rooms, the limits are per-process RAM and CPU on a small VM, so launch N headless clients or measure

connection capacity with idle `ws` connections to the proxy (exercises `worker_connections` and `LimitNOFILE`). Cap `MAX_ROOMS` from measured RSS.

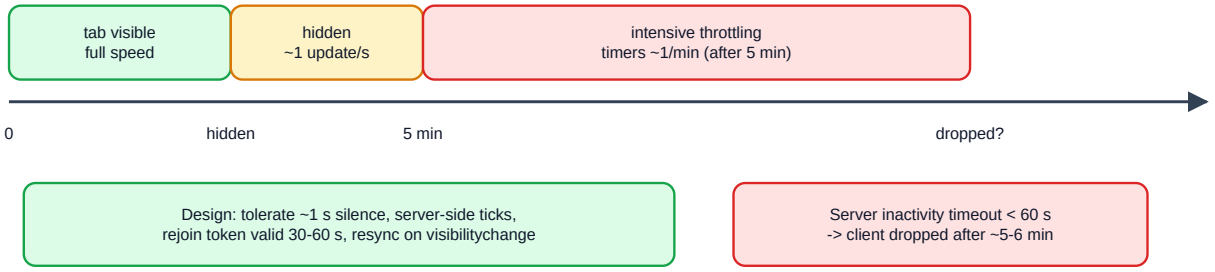
7.4 NGO integration tests

Derive from `NetcodeIntegrationTest` (namespace `Unity.Netcode.TestHelpers.Runtime`), override `NumberOfClients`, use `[UnityTest]` coroutines. The seen source is a 1.2.0 mirror, so verify helper method names against `Packages/com.unity.netcode.gameobjects/TestHelpers/Runtime/` in your project. If the namespace cannot be imported, users report adding the helper package to `testables` in the manifest and referencing it in the test assembly definition (Unity forum).

Proof for Chapter 7: both browser windows join the same room through the VM, see each other's `NetworkVariable` changes, and the smoke commands all behave as annotated.

Chapter 8. Troubleshooting tables

Figure 10. Background tab timeline



Pre-flight checklist

- [ ] DevTools open on Console, Network (filter WS) and the nginx `error.log` / `rooms.log` tailing.
- [ ] You know which layer you are debugging: DNS, TLS, static files, lobby API, WebSocket upgrade, NGO approval, gameplay.

8.1 Connection and TLS

Symptom	Likely cause	Fix	Evidence
Console: "attempted to connect to the insecure WebSocket endpoint ... must be available over WSS"	HTTPS page, transport on `ws://` or raw port 7777	Client `UseEncryption = true`, port 443, proxy in front	cited: Unity forum
`WebSocket connection to 'wss://...' failed`, no detail	Certificate untrusted, expired or wrong host; DNS wrong; port closed; proxy or room down	Open `https://HOST` in a tab; `openssl s_client`; check ufw and (on a cloud VM) the provider security list; `systemctl status`	cert and port causes supported by a Photon thread; rest general knowledge
WS tab shows 400/426/404 on handshake	Wrong path, proxy not sending Upgrade headers, upstream rejecting Origin	Align path and rewrite, check `map` and headers	general knowledge
502 on `/r/70xx`	Room process dead or not yet listening	Check lobby journal; recreate room	inference
101 then closes after about 60 s idle	nginx `proxy_read_timeout` default	Raise timeout; keep traffic flowing	cited: websocket.org
Works on `localhost`, fails after deploy	Mixed content, certificate, CORS or version cache	Re-test over HTTPS; hard refresh; check `Application.version` approval	general

Symptom	Likely cause	Fix	Evidence
Connect fails only on a phone	Phone does not resolve private DuckDNS answer (rebind protection) or does not trust a local CA	Use public DNS resolvers, a router exception, or a public certificate	inference
"Invalid connection type: wss. Must be udp or dtls."	Old NGO/Transport with Relay	Update to NGO 1.6+ / Transport 2.0.2, enable Use Web Sockets, pass `wss` to `SetRelayServerData`	cited: Unity forum (Relay only; not relevant to this stack)
Works in Editor, not WebGL	UDP in use; Use Web Sockets off; WebGL as host	Enable WebSockets on both ends; dedicated server	cited: NGO #4111
Server disconnects client immediately with a reason	Approval rejected: bad token, expired, reused, wrong room, full	Read `NetworkManager.DisconnectReason`; check clock skew between lobby and server (tokens last 60 s)	this guide

8.2 Build and static files

Symptom	Likely cause	Fix	Evidence
"Unable to parse Build/*.framework.js.br (or .gz) ... file is corrupt or compression was misconfigured"	Missing or wrong `Content-Encoding`, or double compression	Add the per-extension locations; `gzip off`; or switch Compression Format; inspect headers in DevTools	cited: Unity forum 305236, 247939
"HTTP response header Content-Type configured incorrectly for file Build/xxx.wasm.gz"	`.wasm` not served as `application/wasm`	`default_type application/wasm` in the `.wasm.gz` location	cited (title only): Unity forum
Brotli build fails on a host	Host lacks Brotli support (itch.io) or the page is not HTTPS	Use Gzip	itch.io reports: thread; HTTPS-only claim is from memory
Old behaviour after redeploy	Browser cache; IndexedDB data cache	`no-cache` on `index.html`; hashed file names; clear site data	general
Cross-origin `UnityWebRequest` works in Editor, fails in WebGL	Missing CORS or bad OPTIONS handling	Same origin via nginx (this design), or correct CORS incl. preflight	cited: Unity docs
Out of memory / wasm compile errors	Heap too small or too large (mobile Safari)	Tune initial/max memory; reduce allocations	general; no source
Server binary "Permission denied"	Missing execute bit after rsync	`chmod +x` (deploy script does it)	general
Linux IL2CPP server link error	Known Unity issue UUM-121029	Use Mono	Issue Tracker (summary)

8.3 Certificates and DNS

Symptom	Likely cause	Fix
certbot DNS-01 times out	Propagation too short or DuckDNS token wrong	`--dns-duckdns-propagation-seconds 60` or more; check credentials file permissions 600
"module not registered: dns.providers.duckdns"	Stock Caddy binary	Build with `xcaddy --with github.com/caddy-dns/duckdns`
Cannot get apex and wildcard together	Single TXT record at DuckDNS	Issue separately
Rate-limit errors	Re-issuing repeatedly; deleting `/etc/letsencrypt`	Use `--staging` while iterating; keep the certificate directory
Cert renewed but nginx still serves the old one	No reload hook	Deploy hook `systemctl reload nginx`
IP certificate fails on home VM	Port 80/443 not reachable from the internet	Not possible behind NAT/CGNAT; use DuckDNS DNS-01

8.4 Process and platform

Symptom	Likely cause	Fix
Lobby exits at start: "TOKEN_SECRET (>=32 chars)..."	Missing env	Fill `/etc/maze/lobby.env`
`room_failed_to_start` (500)	Binary missing, no `+x`, server not listening within 30 s, sandboxing blocked Unity	`journalctl -u maze-lobby`; run the binary by hand as user `maze`; relax hardening options
Rooms reaped after 30 s	Server not sending heartbeats	Check `LOBBY_URL` / `LOBBY_INTERNAL_KEY` env reach Unity and `ServerBootstrap.Heartbeat` runs
Orphan Unity processes after crash	Lobby killed with `SIGKILL` outside systemd	Startup `cleanOrphans` handles pid files; keep `KillMode=control-group`
Server CPU pinned at 100% per room	No frame cap	`Application.targetFrameRate = 30`
Oracle VM stopped by provider	Idle reclamation	Keep real load or upgrade to pay-as-you-go while staying within free limits (Oracle docs: idle if 95th-percentile CPU, network and memory stay under 20% for 7 days)
Host-only quirk: Windows blocks inbound	Windows Firewall	Allow the VM host ports; general knowledge

Chapter 9. Security hardening

Pre-flight checklist

- [ ] SSH key login works from a second session before disabling passwords.
- [ ] `ufw status` shows only 22, 80, 443 (and nothing for 3000 or 7000-7099).
- [ ] Secrets are in `/etc/maze/lobby.env` (mode 640), not in the repo, crontab or shell history.
- [ ] `git log -p | grep -i -E 'token|secret'` is clean.

9.1 Host baseline (UNTESTED)

```
# /etc/ssh/sshd_config.d/10-hardening.conf   (after key login works; run sshd -t before reload)
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
```

Install `fail2ban` with the default `sshd` jail; optionally an `nginx-limit-req` jail reading `/var/log/nginx/error.log` with `bantime = 1h`, `maxretry = 10`. Enable `unattended-upgrades`; consider disabling automatic reboot on a game host. On cloud images, the provider security list and the image's own iptables rules may block traffic independently of ufw.

9.2 Layered controls

Layer	Control	Status
Network	Rooms and lobby bound to loopback; only 22/80/443 open	Lobby binds 127.0.0.1 (TESTED config); the Unity server listens on `127.0.0.1` per `ServerBootstrap` (UNTESTED)
TLS	TLS 1.2 and 1.3 only; generate the full cipher list from the Mozilla "intermediate" profile (profile not fetched)	UNTESTED
Proxy limits	`limit_conn`, `limit_req`, `client_max_body_size 4k`, short header timeouts	In the Chapter 5 config
Handshake Origin	OWASP says check `Origin` on every WebSocket handshake against an allow-list, since browsers set it and scripts cannot override it (OWASP WebSocket cheat sheet, summary). UTP/NGO has no Origin check, so enforce it at nginx.	Snippet below, UNTESTED
Authentication	Lobby issues a 60 s HMAC token bound to room id, player name and a unique id; room validates in `ConnectionApprovalCallback`, constant-time compare, single use	Lobby TESTED; Unity side UNTESTED
Authorization	Every server RPC validates sender, state, range and rate; `InvokePermission = Owner` where appropriate	Pattern in Chapter 3
Process	systemd sandbox, non-root, `MemoryMax`, `TasksMax`, `LimitNOFILE`	UNTESTED

Layer	Control	Status
Secrets	Env vars, never argv; DuckDNS token is an account secret, root-only file, rotate if leaked	Lobby TESTED
Logs	Do not log tokens; nginx logs URLs, so keep secrets out of query strings (this design uses the approval payload)	design

Origin allow-list at nginx for the room location (UNTESTED; Origin can be forged by non-browser clients, so this stops hijacking, not bots):

```
map $http_origin $origin_ok {
    default 0;
    ""      1;                                  # non-browser clients send none; tighten if you
want browsers only
    "https://maze.duckdns.org" 1;
}
# inside location ~ ^/r/(?<room_port>70[0-9][0-9])$ { ... }
#     if ($origin_ok = 0) { return 403; }
```

9.3 Server-authority rules

Validate every `[Rpc(SendTo.Server)]`: range, rate, state, cooldown, finite floats, array and string lengths; take identity from `rpcParams.Receive.SenderClientId`; never trust client position or score; move on the server; cap RPCs per second per client and disconnect offenders; limit players per room and rooms per user. Remember that `RpcInvokePermission` defaults to `Everyone` (inferred), so any client can call any server RPC unless you restrict it.

9.4 Denial of service

A small VM cannot absorb real DDoS. Cap rooms by RAM (`MAX_ROOMS`), keep `limit_conn` at 10-20 per IP but not lower (carrier-grade NAT and schools share addresses), and avoid logging IPs longer than needed. A free Cloudflare proxy in front was not verified for arbitrary WebSocket traffic; Cloudflare Quick Tunnels are limited to 200 in-flight requests with no SLA (Cloudflare docs), so they are a poor fit. ngrok's free plan caps data at 1 GB per month (ngrok).

Chapter 10. CI/CD

Pre-flight checklist

- [ ] Repo has `lobby/package-lock.json` committed.
- [ ] A dedicated deploy SSH key exists whose server-side `authorized_keys` entry is restricted, and a sudoers rule limited to `systemctl restart maze-lobby`.
- [ ] `DEPLOY_HOST_KEY` captured once with `ssh-keyscan -t ed25519 HOST` (pinned, not scanned at runtime).
- [ ] You accept that Unity builds stay local (below).

10.1 Position on Unity builds in CI

GameCI documents the Personal route using a `.ulf` and secrets `UNITY_LICENSE`, `UNITY_EMAIL`, `UNITY_PASSWORD` (GameCI activation), but Unity reportedly no longer supports manual activation of Personal licenses (Unity forum). GameCI cli PR #246 states a `.ulf` can no longer be obtained on a free seat and proposes a "personal" strategy; its merge status is unconfirmed (PR #246). Unity's docs say Personal users activate through Unity Hub, and serial-based command-line activation works only for Plus/Pro (Hub licenses). Workarounds exist and are fragile (DEV Community). **Recommendation: build Unity locally with `BuildScript` (Chapter 4), upload by rsync, and use CI only for the lobby.**

10.2 Lobby test and deploy workflow (UNTESTED)

The lobby has no lint or test scripts yet; add `npm run lint` and `npm test` (or remove those steps). Pin actions to a major version or SHA and verify current majors.

```yaml
# .github/workflows/ci-deploy.yml
name: ci-deploy
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  lobby:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: lobby
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: lobby/package-lock.json
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test --if-present

  deploy:
    needs: lobby
```

```
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - name: Install deploy key and pinned host key
        env:
          KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          HOSTKEY: ${{ secrets.DEPLOY_HOST_KEY }}
          HOST: ${{ secrets.DEPLOY_HOST }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$KEY" > ~/.ssh/deploy_key
          chmod 600 ~/.ssh/deploy_key
          printf '%s %s\n' "$HOST" "$HOSTKEY" > ~/.ssh/known_hosts   # HOSTKEY format: "ssh-ed25519
AAAA..." without hostname
      - name: Rsync lobby
        run: |
          rsync -az --delete --exclude node_modules --exclude .env \
            -e "ssh -i ~/.ssh/deploy_key -o StrictHostKeyChecking=yes" \
            lobby/ ${{ secrets.DEPLOY_USER }}@${{ secrets.DEPLOY_HOST }}:/opt/maze/lobby/
      - name: Install deps and restart
        run: |
          ssh -i ~/.ssh/deploy_key -o StrictHostKeyChecking=yes \
            ${{ secrets.DEPLOY_USER }}@${{ secrets.DEPLOY_HOST }} \
            'cd /opt/maze/lobby && npm ci --omit=dev && sudo systemctl restart maze-lobby'
```

Notes: `webfactory/ssh-agent` only sets up known_hosts for github.com, so the pinned-key approach above avoids it; `burnett01/rsync-deployments` is an alternative action. Public repositories get free hosted-runner minutes; private-repo quotas were not verified. Never store the DuckDNS token or `lobby.env` in the repo.

10.3 Optional GameCI job (UNVERIFIED for Personal licenses in 2026)

```
  unity-webgl:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { lfs: true }
      - uses: actions/cache@v4
        with:
          path: Library
          key: Library-WebGL-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}
      - uses: game-ci/unity-builder@v4
        env:
          UNITY_LICENSE: ${{ secrets.UNITY_LICENSE }}
          UNITY_EMAIL: ${{ secrets.UNITY_EMAIL }}
          UNITY_PASSWORD: ${{ secrets.UNITY_PASSWORD }}
        with:
          targetPlatform: WebGL
          buildMethod: BuildScript.BuildWebGL     # optional; use the Chapter 4 script
      - uses: actions/upload-artifact@v4
        with: { name: Build-WebGL, path: build/WebGL }
```

A Linux server job would use `targetPlatform: StandaloneLinux64` with `customParameters: -standaloneBuildSubtarget Server` and `buildMethod: BuildScript.BuildLinuxServer`; whether GameCI has Unity 6.6 images was not checked.

Chapter 11. Pitfalls checklist

Pre-flight checklist (run before declaring done)

- [ ] Spike passed (Chapter 6).
- [ ] Everything survives `sudo reboot` and `kill -9` of the lobby.
- [ ] Hidden-tab drill done in Chrome, Firefox and Safari (no current Firefox/Safari documentation was retrieved, so test empirically).

Pitfall	What happens	Prevention
`SetConnectionData` after setting the path	Path silently resets to `/`	Always set `WebSocketPath` last
NGO older than 2.11.0	No `WebSocketPath` field	Pin 2.13.3 and check `packages-lock.json`
nginx `add_header` in a location	Drops inherited headers	Repeat needed headers per location
Double compression	Unity "file is corrupt" error	`gzip off`; one compressor only
Staging certificate in browser	Silent WebSocket failure	Switch to production certificate; accept cert in a tab when using self-signed
Deleting `/etc/letsencrypt` or Caddy's data dir between redeploys	Hits the 5 duplicate-certificates-per-week limit	Keep the directory
WebGL is IL2CPP only; stripping removes reflection-reached code	RPCs or types missing at runtime (no NGO-specific reports found, so unconfirmed)	`link.xml`, Low/Medium stripping, test the real builds
Client/server version skew from cached builds	Mysterious failures	Version in approval; `no-cache` on `index.html`
Time zones and `DateTime.Now`	Wrong expiry logic	UTC and epoch seconds only; tokens use epoch seconds
Oracle idle reclamation	VM stopped	Real load or pay-as-you-go within free limits
ARM VM with x86_64 Unity binaries	Cannot execute	Use an x86_64 VM
VirtualBox NAT with no forwards	Nothing reachable	Forward 22/80/443 or use bridged
Wi-Fi bridging flaky on laptops	VM loses network	Use NAT with forwards or a wired adapter (general experience)
Testing only on `localhost` over http	Production TLS and mixed-content issues hidden	Always run the final test over HTTPS on the VM
Restarting the lobby kills rooms	Players dropped on deploy	Drain first, or accept for a prototype
`proxy_read_timeout` at 60 s	Idle rooms cut	3600 s on the room location
Per-IP limits too low	Schools and mobile carriers blocked	10-20 connections per IP minimum

Pitfall	What happens	Prevention
Assuming Colyseus/Mirror would be easier for paths	They are not (Mirror has no path setting)	Stay with NGO >= 2.11

Chapter 12. Diagram index

#	Diagram	Shows	Notes for the illustrator
1	System architecture	Browser, internet, VM boundary, nginx :80/:443, static folder, lobby :3000, room processes :7000-7099, DuckDNS and Let's Encrypt outside the VM	Mark loopback-only components inside a dashed box; mark the only open ports (22/80/443)
2	Certificate issuance (DNS-01)	certbot, DuckDNS TXT update, Let's Encrypt validation, private A record	Emphasize that Let's Encrypt never connects to the VM
3	Request flow: create and join a room	Sequence: POST /api/rooms, port allocation, spawn, ready poll, token response, WSS connect `/r/7003`, nginx rewrite to `/`, approval, heartbeat	Annotate where the path is rewritten and where the HMAC is checked
4	Room lifecycle state machine	starting, ready, closing, gone; transitions: spawn, TCP ready, heartbeat lost, empty past grace, API delete, SIGTERM, exit	Include timers: 30 s ready timeout, 30 s heartbeat timeout, 60 s empty grace, 5 s SIGKILL escalation
5	Token anatomy	`base64url(payload).base64url(HMAC)`, claims r/n/j/exp, TTL 60 s, verified in the Unity server	Show secret flowing lobby to room via environment
6	nginx location matching	Which location handles `/`, `/api/`, `/internal/`, `/r/70xx`, `*.wasm.gz`	Decision tree
7	Build pipeline	One Unity project to WebGL build and Linux Server build, rsync to `/opt/maze/web` and `/opt/maze/server`	Show the version stamp joining both
8	Network modes	NAT with forwards versus bridged, and where DuckDNS points in each	Two side-by-side panels
9	Spike decision tree	Outcomes of Chapter 6 matrix leading to proceed, Plan B or Plan C	
10	Background-tab timeline	Visible, hidden 1 s throttling, intensive throttling, reconnect grace window	Time axis in minutes
11	Fallback routing comparison	Path routing vs port range vs Node router	Show client URL for each

Chapter 13. Claims ledger: what is proven, what is not

Claim	Status	Basis
NGO 2.11.0 added `ConnectionData.WebSocketPath`	**Verified**	NGO CHANGELOG
Default path `/`; applied via `WithWebSocketParameters` only when `UseWebSockets` and not Relay	**Verified**	`UnityTransport.cs`
`SetConnectionData` does not preserve the path	**Verified by code reading, not compiled**	`UnityTransport.cs`
Servers accept only the configured path	Docs summary	Transport manual (summary)
WebGL plugin transmits the path	**Unproven**	Closed-source transport; spike
Proxy-terminated TLS is transparent to UTP frames	**Unproven**; one contrary third-party report	Spike; PlayFab #9
Client `UseEncryption = true` yields `wss` while server uses `false`	**Unproven**	Spike
`duckdns.org` is on the Public Suffix List	**Verified**	PSL raw file
DNS-01 certificates work with a private A record	Community reports	Caddy community
certbot-dns-duckdns command and credentials format	**Verified**	PyPI
acme.sh `DuckDNS_Token` variable	**Verified**	`dns_duckdns.sh`
IP certificates GA 15 Jan 2026, 160 h, http-01/tls-alpn-01 only	Search summaries and secondary sources	LE announcement (not read directly)
Lobby behaviour (create/list/join/validation/reaper/SIGTERM)	**Tested** with a stand-in server on Node 22.22.0	Run in sandbox
Lobby with the real Unity binary	**Untested**	
nginx, Caddy, systemd, VBoxManage, deploy script, CI YAML, all C#	**Untested**	No tools in sandbox
`RpcInvokePermission` default is `Everyone`	Inferred	Enum ordering
Headless room memory 100-250 MB	**Estimate**	Anecdotes only
GameCI with Personal licenses in 2026	**Unresolved**	Conflicting sources
Unity 6000.6.6f1 compatibility with NGO 2.13.3	**Unverified**	Declared minimum is 6000.0

Source-access limits worth knowing: docs.unity3d.com, Unity Discussions, letsencrypt.org, virtualbox.org, Oracle docs, caddyserver.com, duckdns.org, developer.chrome.com and the OWASP cheat-sheet host were unreachable, so every claim attributed to them is a search summary. GitHub raw files, the Public Suffix List, PyPI and the npm registry were read directly.

Conclusion

The central change in understanding is that the one-WSS-port design stopped being a hope and became an engineering task with a defined test: the missing piece was a client-side path, and NGO has offered it since 2.11.0, together with the discovery that the same property that makes paths possible, the server only accepting its configured path, is solved by a one-line `rewrite` in the proxy. What remains is narrow and empirical. Because the transport plugin is closed-source and the research environment had no Unity Editor, whether the browser actually sends the path and whether the proxy leaves frames intact can only be known by running Chapter 6. The guide is written so that a failed spike costs little: the port-range generator and the Node upgrade router are tested or ready-to-test drop-ins that leave the lobby, token scheme, systemd units and Unity scripts unchanged.

Two practical implications follow. First, certificate trust, not code, is the likeliest source of first-week failures, and DuckDNS with DNS-01 is the only sourced route to a browser-trusted certificate for a VM that is not on the public internet, which is why the VirtualBox NAT or bridged choice does not affect HTTPS at all. Second, the highest-value measurements still missing are cheap to take on the real VM: RSS and CPU per room (to set `MAX_ROOMS`), the hidden-tab survival time per browser (to set heartbeat timeouts and the reconnect grace), and whether SIGTERM produces a clean Unity shutdown. Record those three numbers in the ledger in Chapter 13 and the guide moves from a well-sourced draft to a verified runbook.